

GPIO 基础原理与 EXIT 实践

Lecture 5

预习资料:

必看:

【【开漏? 推挽? 都是些什么鬼? 动画帮你一网打尽】 【STM32入门教程-2024】 第5集 关于STM32 GPIO内部的那些事】

https://www.bilibili.com/video/BV1zG4y1K78S/?share_source=copy_web&vd_source=ea9c602d1826d7882b8e5957fb67d8b3

【【keysking的STM32教程】 第7集 深入讲解STM32中断】

https://www.bilibili.com/video/BV1M24y1473t/?share_source=copy_web&vd_source=ea9c602d1826d7882b8e5957fb67d8b3

【【keysking的stm32教程】 第6集 狂飙STM32中断】

https://www.bilibili.com/video/BV1Fj411V7aq/?share_source=copy_web&vd_source=ea9c602d1826d7882b8e5957fb67d8b3

选看:

【【keysking的STM32教程】 第8集 STM32的串口通信】

https://www.bilibili.com/video/BV1Na4y1T7VQ/?share_source=copy_web&vd_source=ea9c602d1826d7882b8e5957fb67d8b3

【【STM32入门教程-2025】 第9集 STM32串口原理与串口中断模式收发 | keysking的stm32教程】

https://www.bilibili.com/video/BV1bc411J7Tv/?share_source=copy_web&vd_source=ea9c602d1826d7882b8e5957fb67d8b3

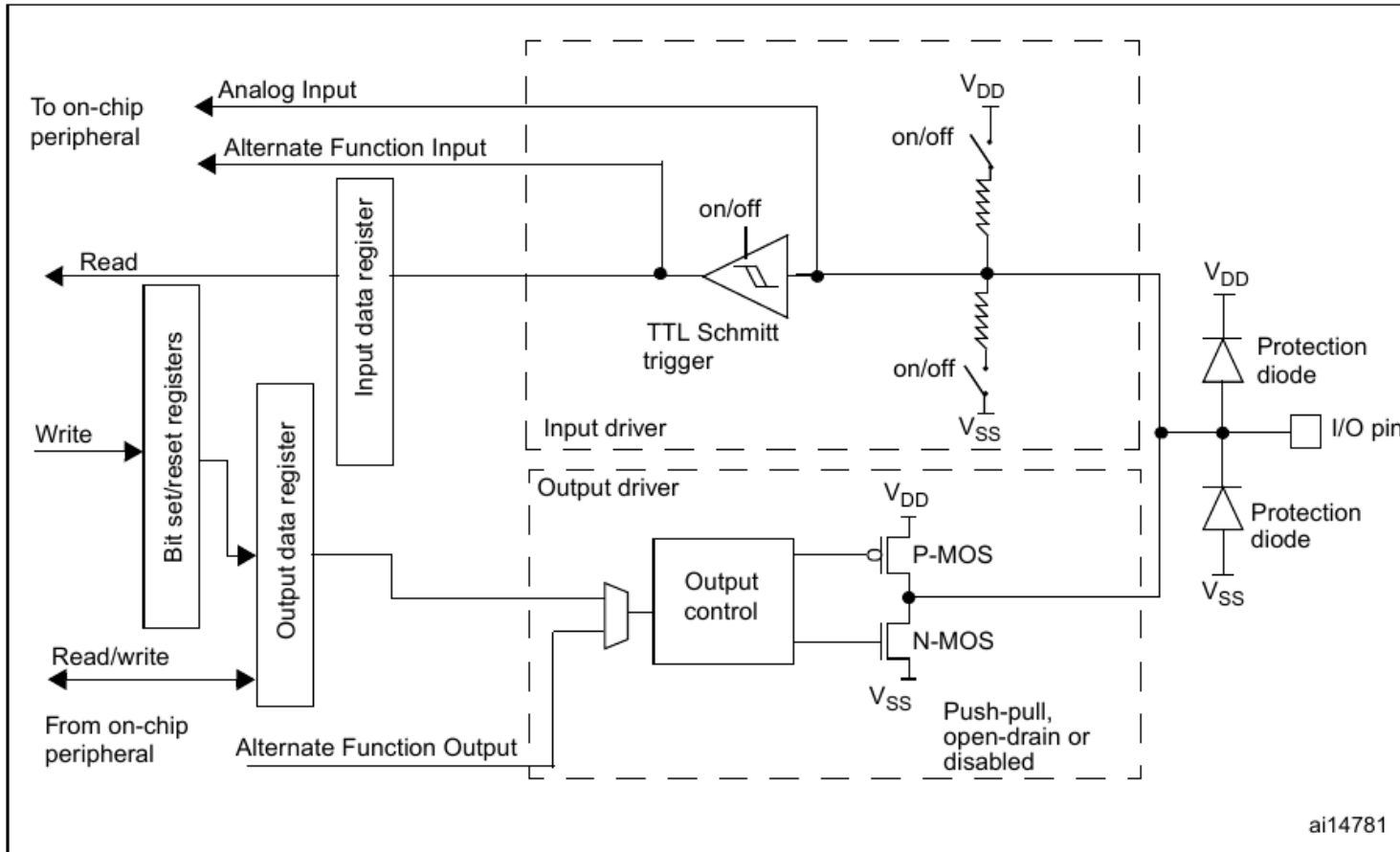
预习须知：

预习把握大体思路即可，如果课件有不明白的部分**也没有问题。**

很多点有印象即可，**不需要记住每个寄存器，
重在体会单片机和HAL库的设计逻辑**

GPIO 是什么?

Figure 13. Basic structure of a standard I/O port bit



General Purpose Input / Output

- 同一根管脚，既能读外部电平，也能驱动外部负载
- 输入、输出、复用功能不是三套引脚，而是同一套内部开关
- 配置寄存器的本质：决定哪些开关接通

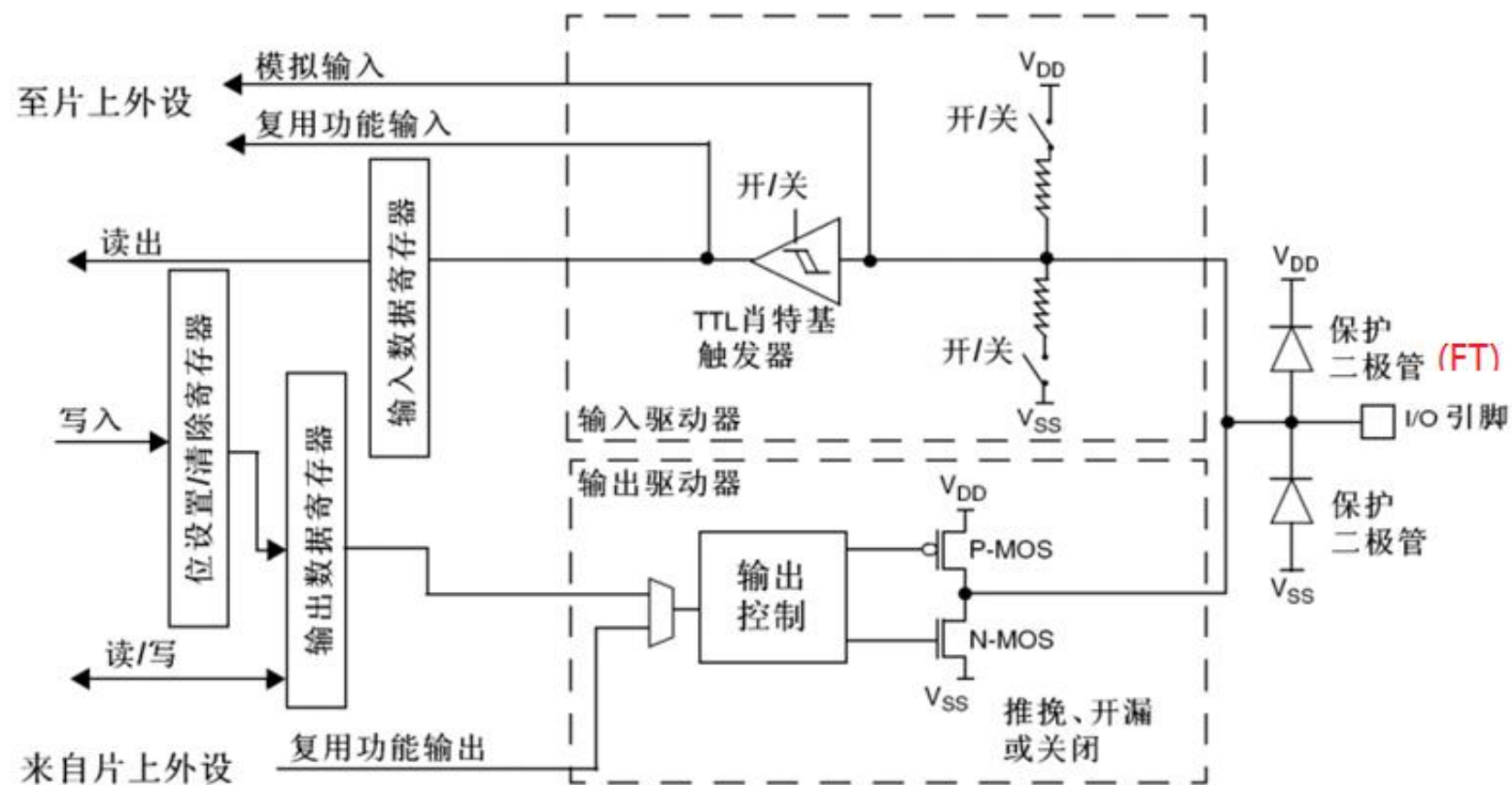
找到右侧 I/O pin，从右向左看

视频导入：MOS 管的导通与截止



► [点击此处播放《MOS管的工作原理》（Bilibili，约 4 分 58 秒）](#)

一张图看懂一根 GPIO



① 保护电路

钳位异常电压，保护芯片内部

② 输入驱动

施密特触发器把电压整理成数字电平

③ 输出驱动

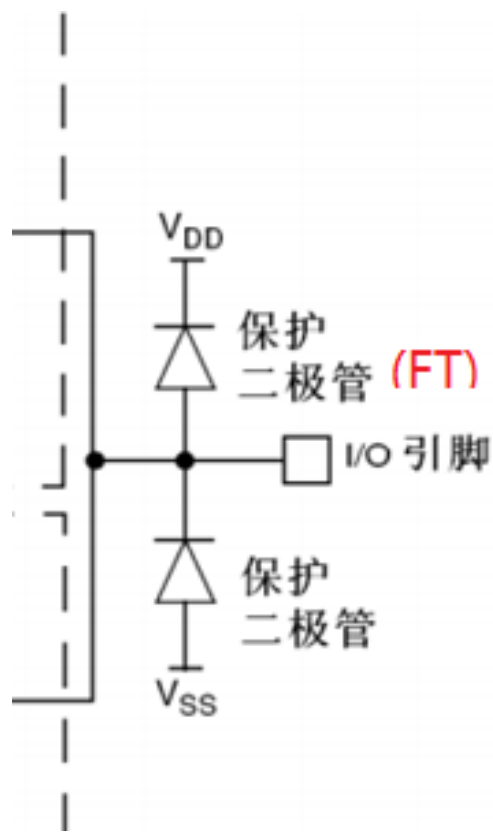
P-MOS 与 N-MOS 决定推挽或开漏

④ 数据/复用

寄存器和片上外设决定数据来源

图中寄存器名：BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

保护二极管与 5V 容忍引脚



标准 I/O 的钳位路径

电压高于 VDD 或低于 VSS 太多时，钳位通道开始导通。保护二极管只承受异常和瞬态。

I_{lkg}	Input leakage current ⁽⁶⁾	$V_{SS} \leq V_{IN} \leq V_{DD}$ Standard I/Os	-	-	± 1	μA
		$V_{IN} = 5 V$ I/O FT	-	-	3	
R_{PU}	Weak pull-up equivalent resistor ⁽⁷⁾	$V_{IN} = V_{SS}$	30	40	50	k Ω
R_{PD}	Weak pull-down equivalent resistor ⁽⁷⁾	$V_{IN} = V_{DD}$	30	40	50	
C_{IO}	I/O pin capacitance		-	5	-	pF

- 1. Data based on design simulation.
- 2. Tested in production.
- 3. FT = 5 V tolerant. To sustain a voltage higher than $V_{DD} + 0.3 V$ the internal pull-up/pull-down resistors must be disabled.
- 4. Hysteresis voltage between Schmitt trigger switching levels. Evaluated by characterization, not tested in production, unless otherwise specified.
- 5. With a minimum of 100 mV.

FT = 5 V tolerant

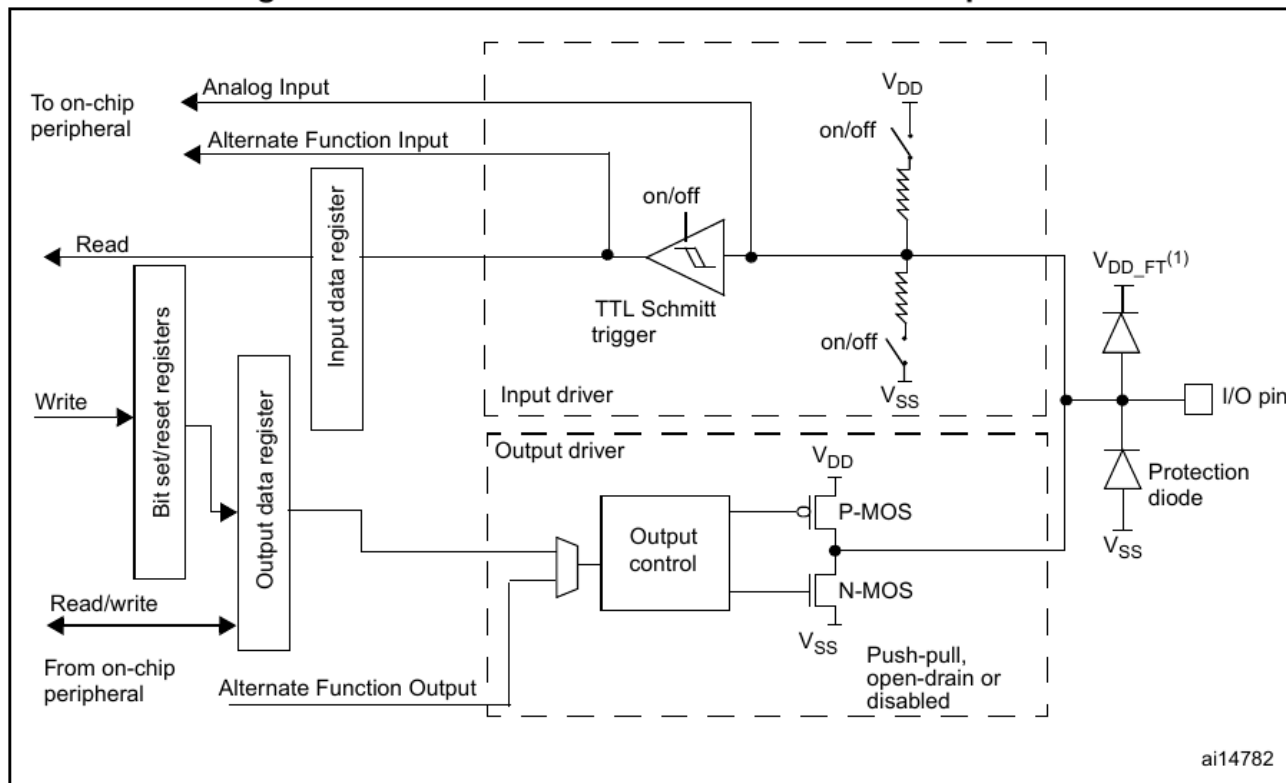
先查数据手册的 I/O level。STM32F103x8/xB 中，PB6 标为 FT，PA0 没有 FT 标记。

- VIN 高于 VDD + 0.3 V 时，内部上下拉必须关闭。HAL 使用 GPIO_NOPULL
- FT 只描述数字输入耐压。输出高电平仍由 VDD 决定
- 模拟功能和非 FT 引脚必须遵守各自的电压极限

先确认 FT 标记，再检查引脚功能和上下拉条件。

5V 容忍不等于所有情况下都能接 5V

Figure 14. Basic structure of a 5-Volt tolerant I/O port bit



1. V_{DD_FT} is a potential specific to 5-Volt tolerant I/Os, and different from V_{DD} .

- 只有数据手册标注 FT 的引脚才具备 5V 容忍能力
- 5V 容忍主要针对数字输入条件，不代表模拟模式也允许 5V
- 芯片未供电时，外部高电平还可能形成反向供电路径

Selector

Cross Selector

Features

Block Diagram

Docs & Resources

CAD Resources

Datasheet

Buy

Start Project

STM32F1 Series

STM32F103C8T6

ACTIVE

Product is in mass production

Mainstream Performance line, Arm Cortex-M3 MCU with 64 Kbytes of Flash memory, 72 MHz CPU, motor control, USB and CAN

Unit Price for 10kU (US\$) : 2.8462

LQFP 48 7x7x1.4 mm

The STM32F103xx medium-density performance line family incorporates the high-performance Arm® Cortex®-M3 32-bit RISC core operating at a 72 MHz frequency, high-speed embedded memories (Flash memory up to 128 Kbytes and SRAM up to 20 Kbytes), and an extensive range of enhanced I/Os and peripherals connected to two APB buses. All devices offer two 12-bit ADCs, three general purpose 16-bit timers plus one PWM timer, as well as standard and advanced communication interfaces: up to two I²Cs and SPIs, three USARTs, an USB and a CAN.

The devices operate from a 2.0 to 3.6 V power supply. They are available in both the -40 to +85°C temperature range and the -40 to +105 °C extended temperature range. A comprehensive set of power-saving mode allows the design of low-power applications.

The STM32F103xx medium-density performance line family includes devices in six different package types: from 36 pins to 100 pins. Depending on the device chosen, different sets of peripherals are included, the description below gives an overview of the complete range of

MCUs/MPUs List: 2 items

Export

Commercial Pa...	Part No	Reference	Marketing ...	Unit Price for 10k...	Board	Package	Flash	RAM	I/O	Frequ...
☆ STM32F103C8T6	STM32F103C8	STM32F103C8Tx	Active	2.8462		LQFP 48 7x7x1.4 mm	64 kBytes	20 kBytes	37	72 MHz
☆ STM32F103C8T6...		STM32F103C8Tx	Active	2.8462		LQFP 48 7x7x1.4 mm	64 kBytes	20 kBytes	37	72 MHz

- 可以在cubemx的芯片选择页面查看芯片手册

寄存器的位域表示法：[高位:低位]

$$\text{CNF6}[1:0] = 10_2$$

CNF6

字段名称

CNF 表示配置位
数字 6 表示第 6 号引脚

[1:0]

位位置范围

从 bit1 读到 bit0
共 $1 - 0 + 1 = 2$ 位

10_2 的含义

bit1 = 1, bit0 = 0; 下标 2 表示二进制。

再看一个例子

CRL[27:24] 包含 bit27、bit26、bit25、bit24, 共 4 位。

读法固定：先高位，后低位。方括号表示位位置，不是 C 语言数组。

GPIO 端口寄存器全景

寄存器	关键位域	作用
GPIOx_CRL	CNFy[1:0] MODEy[1:0]	配置 0 至 7 号引脚的模式、驱动方式与输出速度
GPIOx_CRH	CNFy[1:0] MODEy[1:0]	配置 8 至 15 号引脚的模式、驱动方式与输出速度
GPIOx_IDR	IDRy	读取引脚经过输入缓冲后的实际数字电平
GPIOx_ODR	ODRy	输出模式保存写入值；上拉/下拉输入时选择内部电阻接 VDD 或 VSS
GPIOx_BSRR	BSy / BRy	一次写操作置位或清零 ODR，避免读改写冲突
GPIOx_LCKR	LCKy / LCKK	锁定端口配置，芯片复位后解除

x = GPIO 端口字母 (A ~ G) ； y = 端口内引脚编号 (0 ~ 15) 。例如 GPIOA_ODR 的 ODR5 对应 PA5。

MODE 与 CNF 配置位

MODE

Mode bits (模式位)

先判断引脚是输入还是输出。
作为输出时，MODE 还选择最大输出速度。

CNF

Configuration bits (配置位)

在 MODE 已确定的前提下，继续选择输入接法，
或选择输出的驱动方式与数据来源。

每根引脚使用 4 位配置

高 2 位: CNF_x[1:0] 低 2 位: MODE_x[1:0]

x 表示引脚编号。例如 CNF6 与 MODE6 共同配置一个端口中的第 6 号引脚。

STM32F103 的八种 GPIO 模式

Table 20. Port bit configuration table

Configuration mode		CNF1	CNF0	MODE1	MODE0	PxODR register
General purpose output	Push-pull	0	0	01 10 11 see Table 21	0 or 1	
	Open-drain		1			0 or 1
Alternate Function output	Push-pull	1	0		Don't care	
	Open-drain		1		Don't care	
Input	Analog	0	0	00	Don't care	
	Input floating		1		Don't care	
	Input pull-down	1	0		0	
	Input pull-up				1	

Table 21. Output MODE bits

MODE[1:0]	Meaning
00	Reserved
01	Maximum output speed 10 MHz
10	Maximum output speed 2 MHz
11	Maximum output speed 50 MHz

输入：MODE = 00

- CNF=00：模拟输入
- CNF=01：浮空输入
- CNF=10, ODR=0：下拉输入
- CNF=10, ODR=1：上拉输入

输出：MODE > 00

- CNF=00：通用推挽输出
- CNF=01：通用开漏输出
- CNF=10：复用推挽输出
- CNF=11：复用开漏输出

配置寄存器：CRL 与 CRH

9.2 GPIO registers

Refer to [Section 2.2 on page 45](#) for a list of abbreviations used in register descriptions.
The peripheral registers have to be accessed by words (32-bit).

9.2.1 Port configuration register low (GPIOx_CRL) (x=A..G)

Address offset: 0x00
Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30, 27:26, 23:22, 19:18, 15:14, 11:10, 7:6, 3:2, **CNFy[1:0]**: Port x configuration bits (y= 0 .. 7)
These bits are written by software to configure the corresponding I/O port.
Refer to [Table 20: Port bit configuration table](#).
In input mode (MODE[1:0]=00):
00: Analog mode
01: Floating input (reset state)
10: Input with pull-up / pull-down
11: Reserved
In output mode (MODE[1:0] > 00):
00: General purpose output push-pull

9.2.2 Port configuration register high (GPIOx_CRH) (x=A..G)

Address offset: 0x04
Reset value: 0x4444 4444

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30, 27:26, 23:22, 19:18, 15:14, 11:10, 7:6, 3:2, **CNFy[1:0]**: Port x configuration bits (y= 8 .. 15)
These bits are written by software to configure the corresponding I/O port.
Refer to [Table 20: Port bit configuration table](#).
In input mode (MODE[1:0]=00):
00: Analog mode
01: Floating input (reset state)
10: Input with pull-up / pull-down
11: Reserved
In output mode (MODE[1:0] > 00):
00: General purpose output push-pull
01: General purpose output Open-drain
10: Alternate function output Push-pull
11: Alternate function output Open-drain
Bits 29:28, 25:24, 21:20, 17:16, 13:12, **MODEy[1:0]**: Port x mode bits (y= 8 .. 15)
These bits are written by software to configure the corresponding I/O port.

CRL = Configuration Register Low

低位配置寄存器，负责 Pin 0 ~ 7

CRH = Configuration Register High

高位配置寄存器，负责 Pin 8 ~ 15

这里的 Low / High 是引脚编号分组，不是输出低电平 / 高电平。

示例：PA6 的 4 位配置字段

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

CNF6[1:0]

PA6 字段的高 2 位，决定输入接法或输出类型

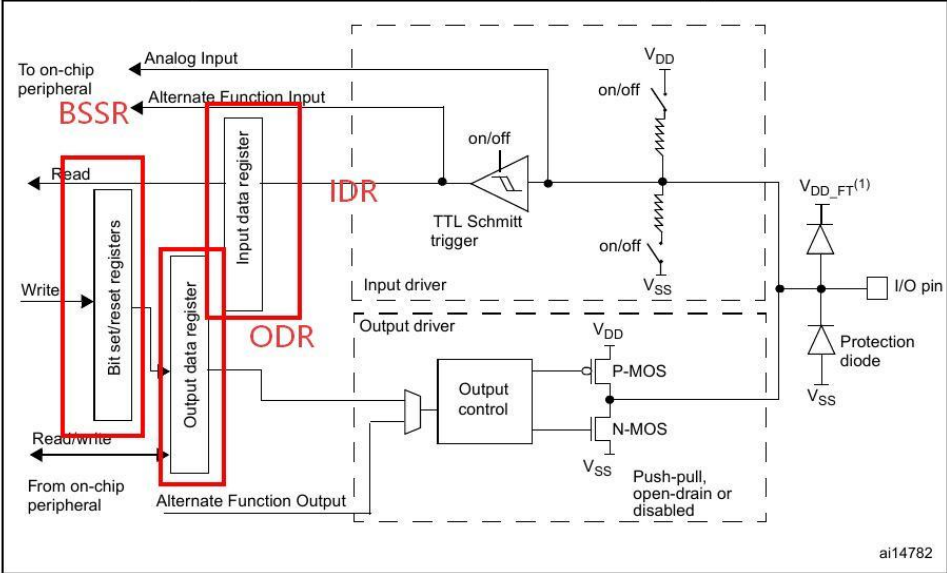
MODE6[1:0]

PA6 字段的低 2 位，先决定输入/输出。

- 1 PA6 的编号是 6，因此查 CRL
- 2 PA6 对应 CRL 的 bit[27:24]
- 3 上拉输入：MODE6=00，CNF6=10
- 4 F103 再用 ODR6=1 选择上拉

IDR 与 ODR:

Figure 14. Basic structure of a 5-Volt tolerant I/O port bit



1. V_{DD_FT} is a potential specific to 5-Volt tolerant I/Os, and different from V_{DD} .

9.2.3 Port input data register (GPIOx_IDR) (x=A..G)

Address offset: 0x08h

Reset value: 0x0000 XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data (y= 0 .. 15)

These bits are read only and can be accessed in Word mode only. They contain the input value of the corresponding I/O port.

9.2.4 Port output data register (GPIOx_ODR) (x=A..G)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data (y= 0 .. 15)

These bits can be read and written by software and can be accessed in Word mode only.

Note: For atomic bit set/reset, the ODR bits can be individually set and cleared by writing to the GPIOx_BSRR register (x = A .. G).

IDR为只读 (read only, 对应图上 “r”) : 当前引脚电平
每次读取都会重新采样。外部电压变了, IDR 也跟着变。

ODR: 被记住的位值
CPU 写入 1 或 0 后会一直保留, 直到再次写入、芯片复位或掉电。

输出模式: ODRy=1/0 决定高/低电平。
上拉/下拉输入: 输出驱动关闭, ODRy=1 使内部电阻接 VDD, ODRy=0 使它接 VSS。

CubeMX: 从引脚角色进入 GPIO 参数

```
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    /* USER CODE BEGIN MX_GPIO_Init_1 */

    /* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOD_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(LED_R_GPIO_Port, LED_R_Pin, GPIO_PIN_RESET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET);

    /*Configure GPIO pin : LED_R_Pin */
    GPIO_InitStruct.Pin = LED_R_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_OD;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(LED_R_GPIO_Port, &GPIO_InitStruct);

    /*Configure GPIO pin : PA10 */
    GPIO_InitStruct.Pin = GPIO_PIN_10;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

    /* USER CODE BEGIN MX_GPIO_Init_2 */

    /* USER CODE END MX_GPIO_Init_2 */
}
```

1

Pinout 中单击引脚

先选择 GPIO_Input、GPIO_Output、Analog 或 EXTI

2

System Core > GPIO

打开 GPIO 配置表, 选中要修改的引脚

3

补齐参数

设置 Mode、Pull、Maximum output speed 和初始电平

4

Generate Code

CubeMX 生成 MX_GPIO_Init() 与 HAL 宏常量

CubeMX 负责填写配置, HAL_GPIO_Init() 负责把配置转换为 CRL、CRH、ODR 和 EXTI 设置。

HAL 宏常量： 界面选项怎样写入 GPIO_InitTypeDef

CubeMX 选项	GPIO_InitStruct.Mode	STM32F1 的含义
Input mode	GPIO_MODE_INPUT	数字输入； Pull 决定浮空、上拉或下拉
Output Push Pull	GPIO_MODE_OUTPUT_PP	通用推挽输出
Output Open Drain	GPIO_MODE_OUTPUT_OD	通用开漏输出
Alternate Function Push Pull	GPIO_MODE_AF_PP	复用推挽输出
Alternate Function Open Drain	GPIO_MODE_AF_OD	复用开漏输出
Analog	GPIO_MODE_ANALOG	模拟输入， 关闭数字输入缓冲
External Interrupt	GPIO_MODE_IT_RISING / FALLING	数字输入， 同时配置 EXTI 触发边沿

Pull: GPIO_NOPULL / GPIO_PULLUP / GPIO_PULLDOWN Speed: GPIO_SPEED_FREQ_LOW / MEDIUM / HIGH

MX_GPIO_Init: 宏常量最终怎样变成初始化代码

```
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure = {0};

    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();

    GPIO_InitStructure.Pin = GPIO_PIN_0;
    GPIO_InitStructure.Mode = GPIO_MODE_INPUT;
    GPIO_InitStructure.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13,
                      GPIO_PIN_SET);
    GPIO_InitStructure.Pin = GPIO_PIN_13;
    GPIO_InitStructure.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStructure.Pull = GPIO_NOPULL;
    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStructure);
}
```

1

先开 GPIO 时钟

没有外设时钟，GPIO 寄存器配置不会生效

2

Pin 选择引脚

GPIO_PIN_0 与 GPIO_PIN_13 是位掩码

3

Mode / Pull / Speed

这些宏对应前面学过的 MODE、CNF、ODR 与速度位

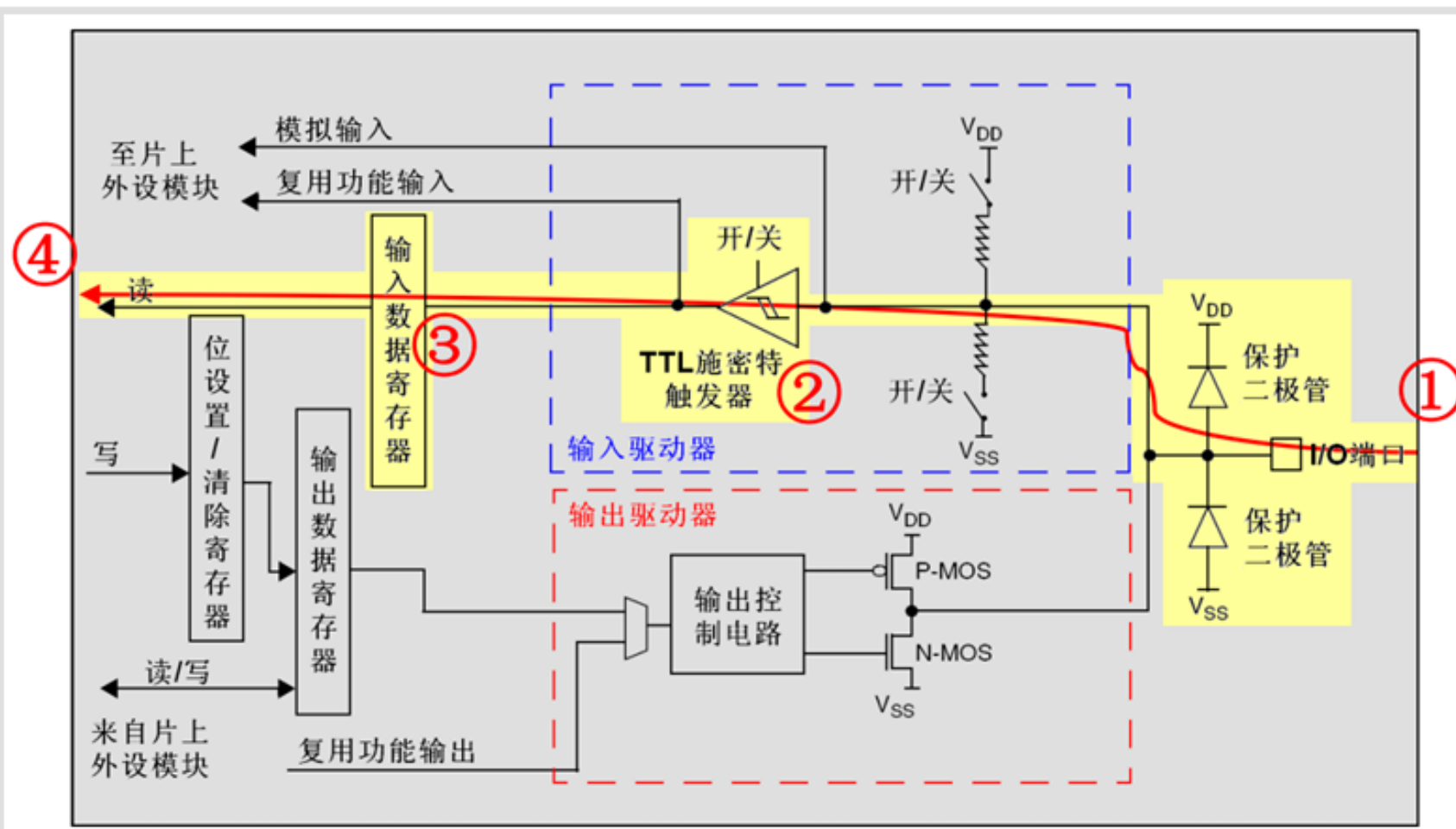
4

调用 HAL_GPIO_Init

HAL 根据引脚号选择 CRL 或 CRH，并写入 4 位字段

学习重点：先看宏表达的意图，再追 HAL 如何把意图写进寄存器。

输入信号经过了什么？



- 外部电压先经过保护二极管
- 施密特触发器把慢变化或带噪声的电压整形为 0/1
- 输入数据寄存器 IDR (Input Data Register) 读到整理后的数字电平
- 同一输入还可送往片上外设或复用输入

问题：输出模式下，还能读取 IDR 吗？
可以。输入通路通常仍然有效。

图中寄存器名：BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

寄存器：程序与功能电路之间的桥梁

1

HAL 函数表达意图

```
HAL_GPIO_WritePin(GPIOA,  
                  GPIO_PIN_5,  
                  GPIO_PIN_SET);
```

函数名便于阅读，也统一处理芯片系列之间的差异。

2

HAL 读写寄存器

```
GPIOA->BSRR =  
    GPIO_BSRR_BS5;
```

寄存器有固定地址。每一位都连接到硬件控制信号或状态信号。

3

功能电路执行动作

BSRR 置位逻辑

ODR 锁存器 = 1

输出级驱动 PA5

HAL 没有绕开寄存器。它把寄存器地址、位掩码和操作顺序封装成易用函数。

施密特触发器：两个阈值形成迟滞区

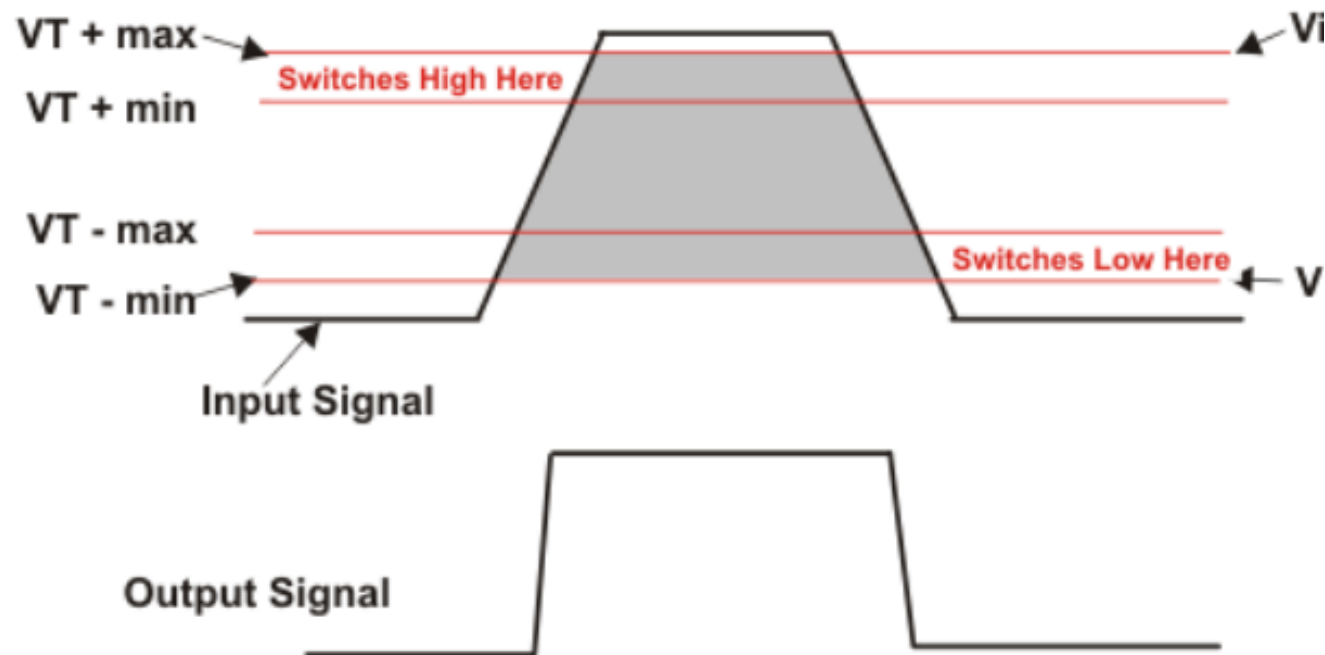


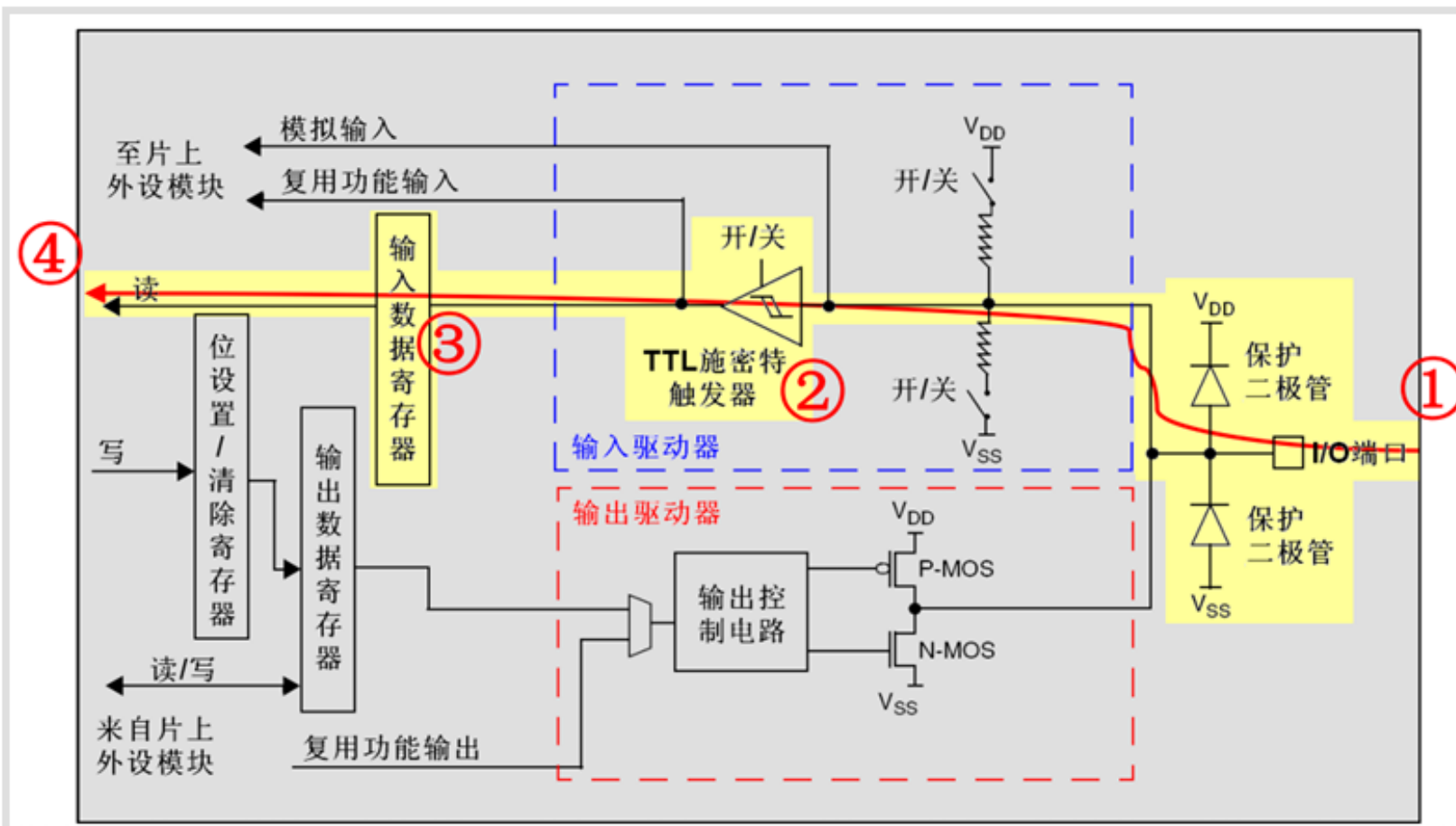
Figure 1. Hysteresis Switch Time

- 输入上升并越过 $VT+$ 后，数字输出切换为 1
- 输入下降并低于 $VT-$ 后，数字输出切换为 0
- $VT-$ 到 $VT+$ 之间保持前一次状态，这段区间称为迟滞
- 小幅噪声落在迟滞区内时，不会造成输出反复跳变

$$VT+ > VT-$$

GPIO 数字输入经过施密特触发器。模拟模式关闭这条数字输入通路，让 ADC 直接采样连续电压

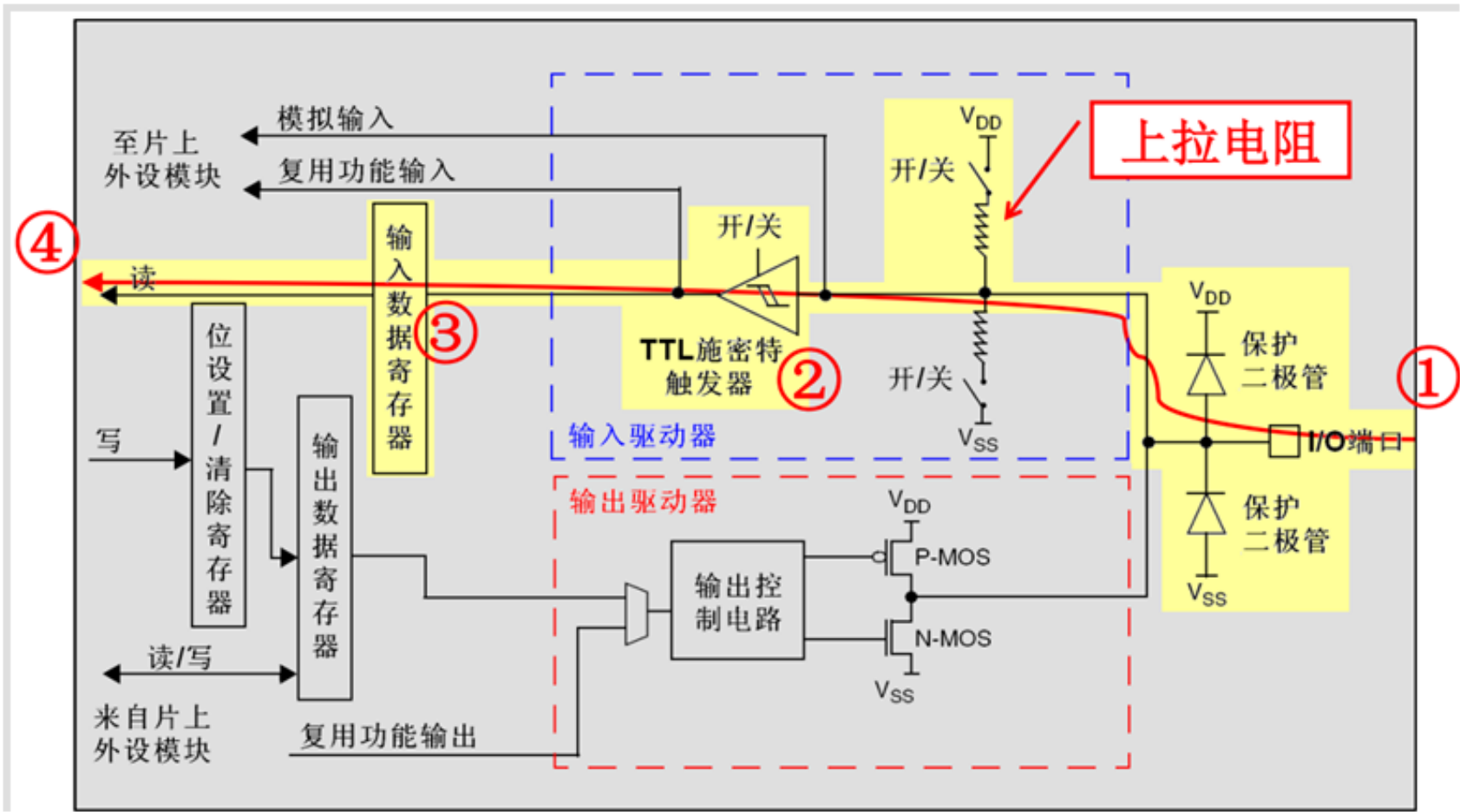
浮空输入：没有确定电平来源



- 编码：MODE=00, CNF=01
- 内部上拉、下拉都断开
- 引脚阻抗高，微小耦合和环境噪声都可能改变输入
- 适合外部电路已经提供明确高/低电平的场景
- 悬空按键输入不是“默认 0”，而是“未知”

图中寄存器名：BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

上拉输入：给未按下状态一个默认值

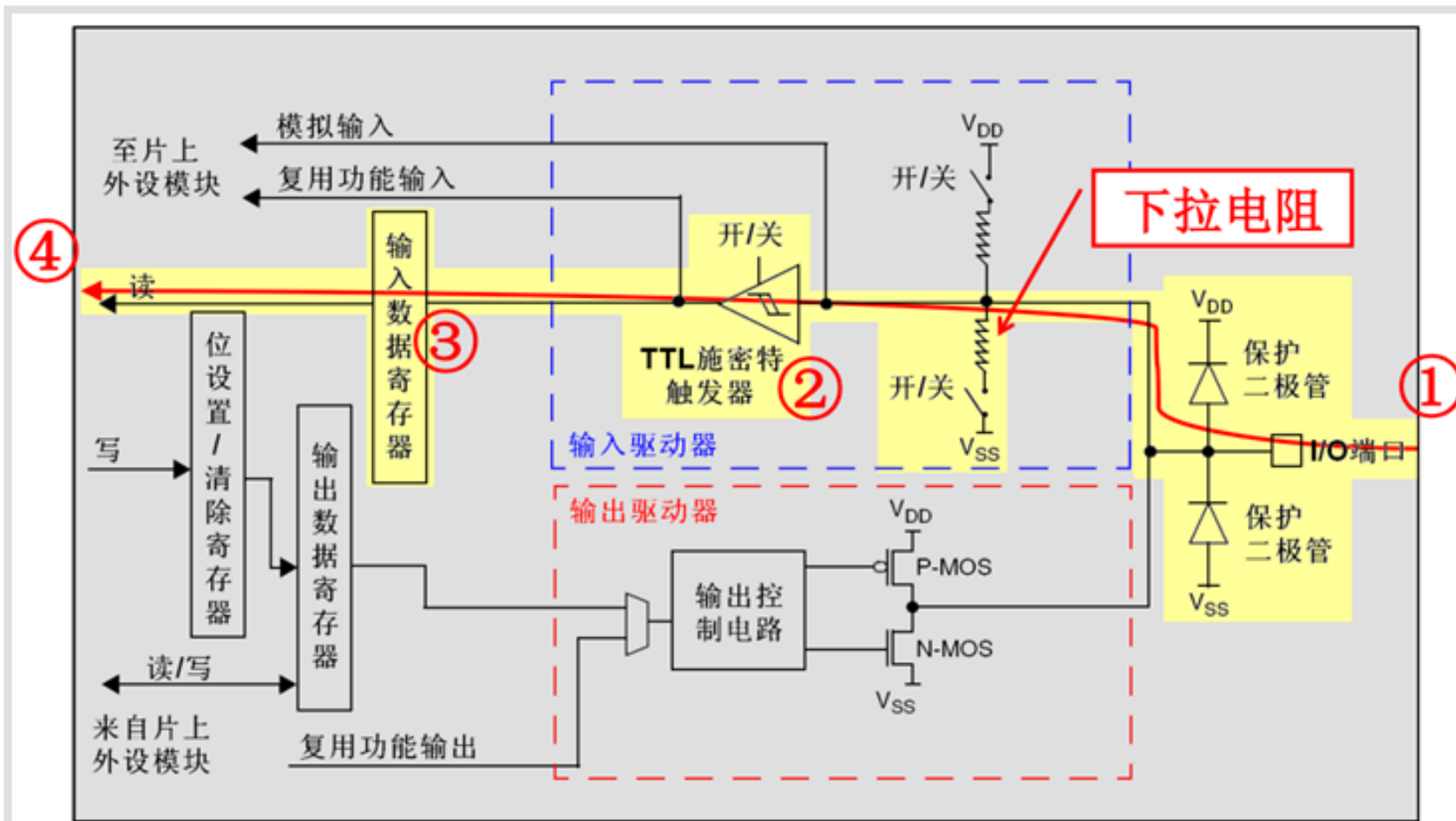


- 编码：MODE=00, CNF=10
ODR=1
- 上拉电阻把空闲电平拉向VDD
- 按键按下接地：引脚从1变为0
- F103 用输出数据寄存器 ODR 的对应位选择上拉
- 软件通常检测下降沿

空闲=1，按下=0

图中寄存器名：BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

下拉输入：给空闲状态定义为 0

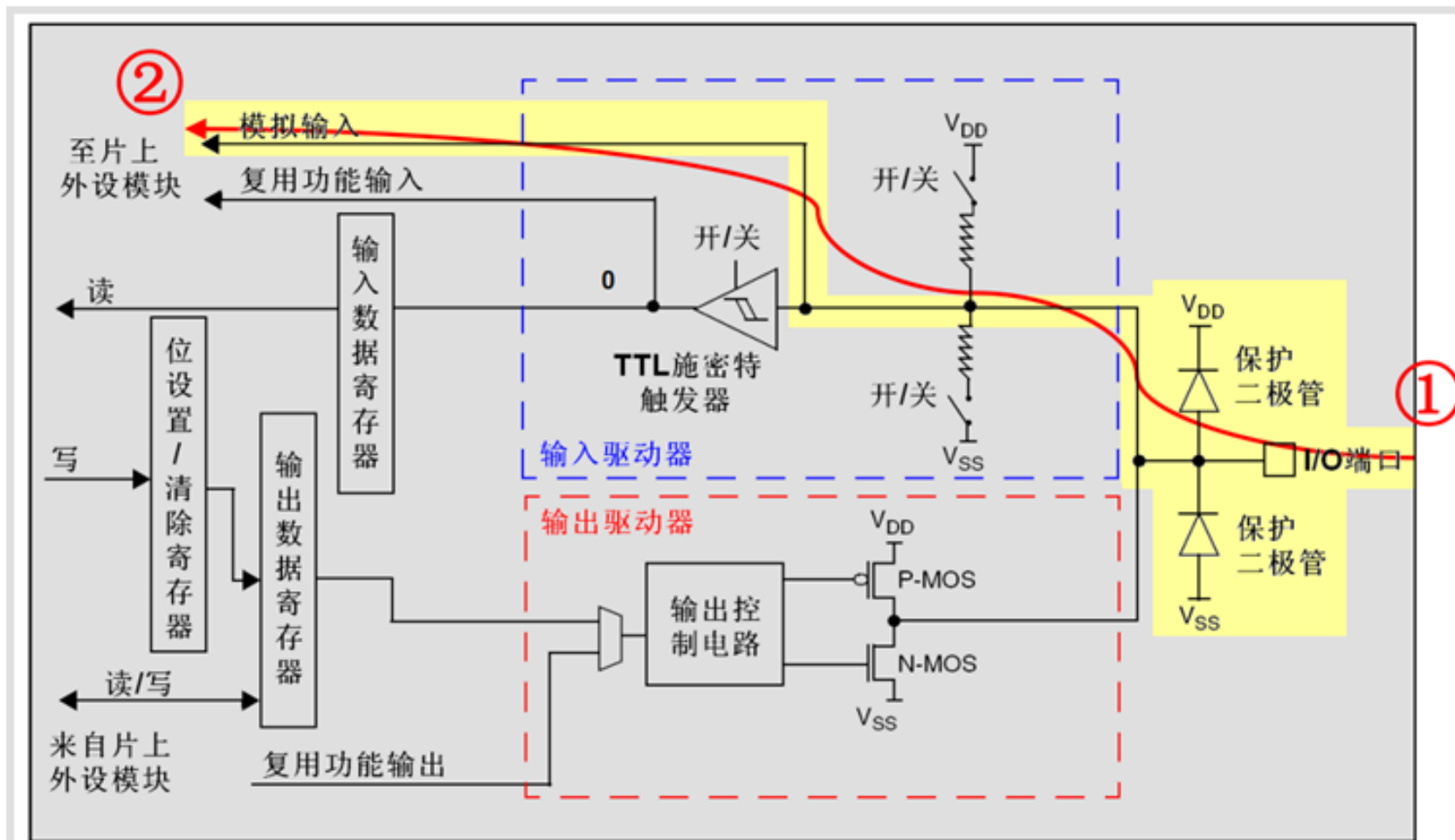


- 编码: MODE=00, CNF=10
ODR=0
- 下拉电阻把空闲电平拉向 VSS
- 按键按下接 VDD: 引脚从 0 变为 1
- F103 用输出数据寄存器 ODR 的对应位选择下拉
- 软件通常检测上升沿

空闲=0, 按下=1

图中寄存器名: BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

模拟输入：绕开数字输入整形

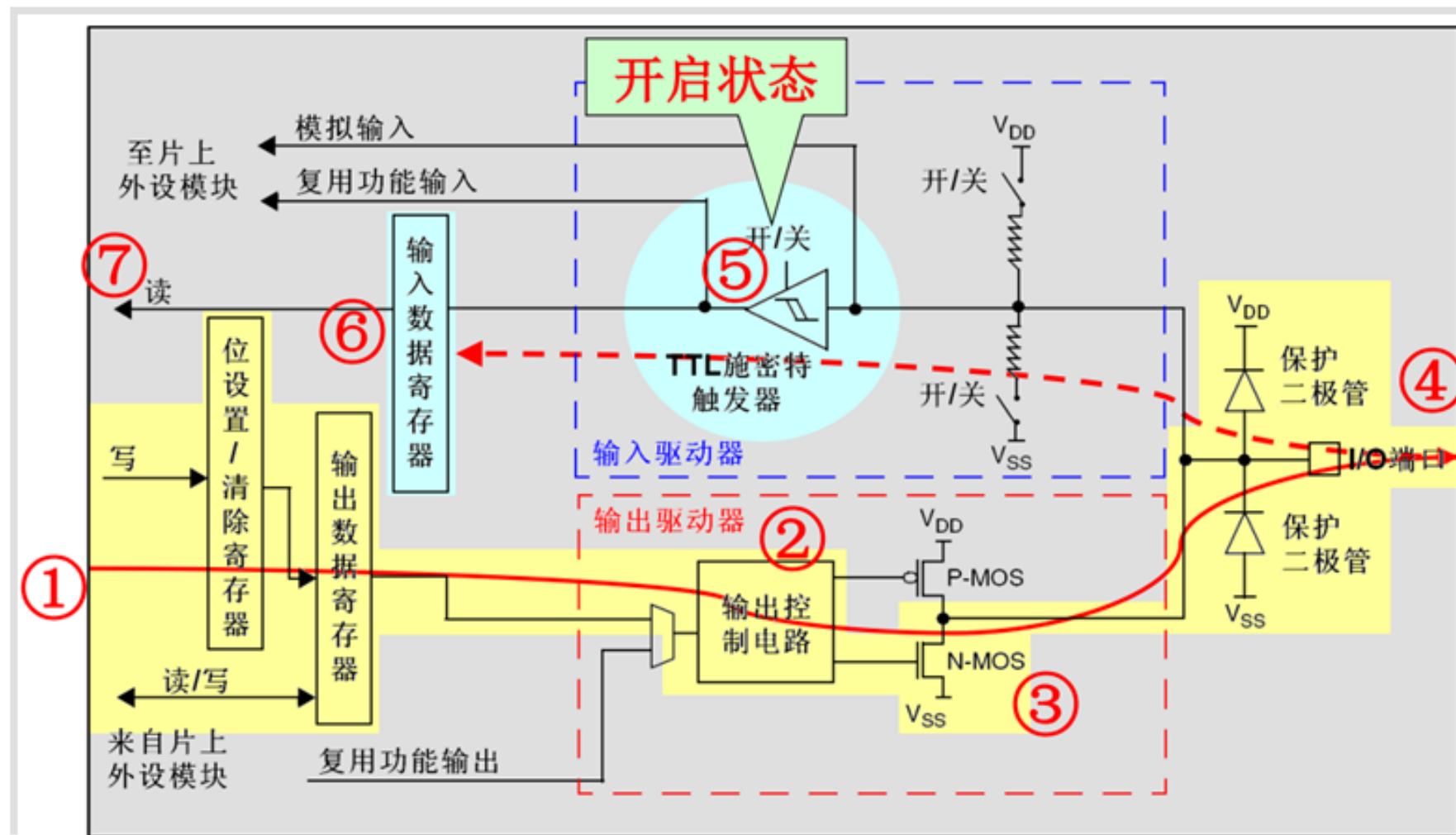


- 编码：MODE=00, CNF=00
- 数字输入中的施密特触发器使用两个阈值：上升越过 V_{IH} 才输出 1，下降低于 V_{IL} 才输出 0
- 两个阈值之间保持原状态，可抑制慢边沿和小幅噪声引起的反复跳变
- 模拟模式关闭数字输入缓冲与上下拉，信号直接送往 ADC

**模拟输入绕开数字整形。
读电压用 ADC，读 0/1 用 IDR**

图中寄存器名：BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

输出信号从哪里来?

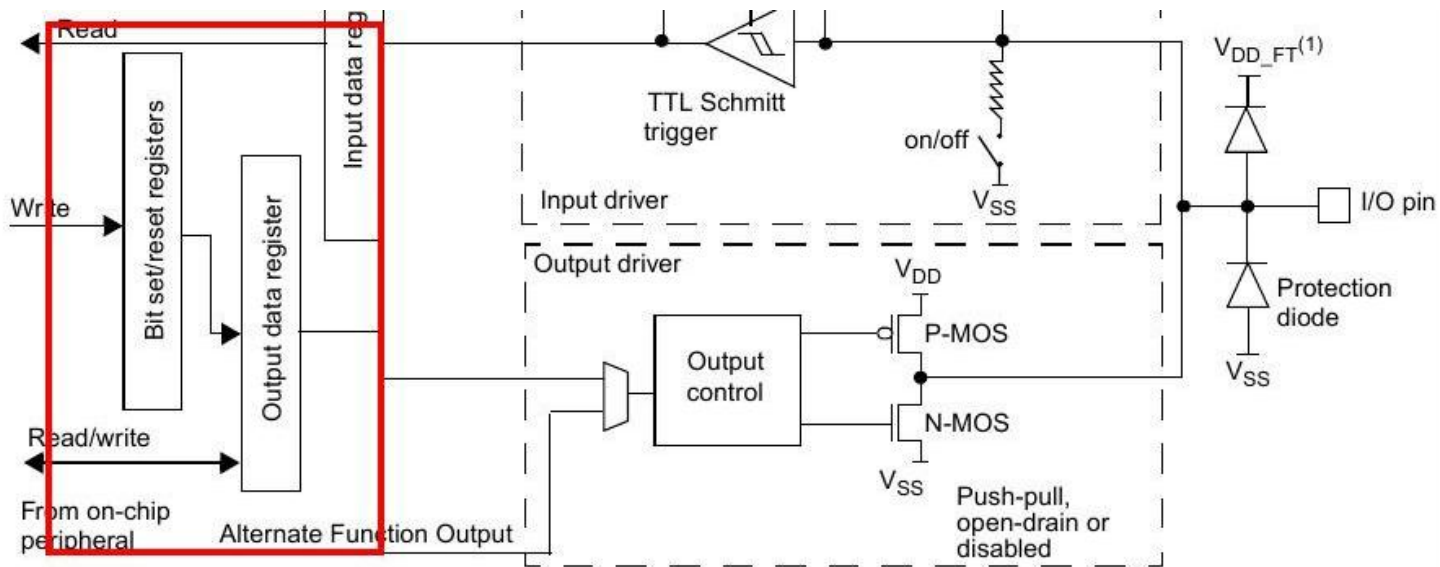


- CPU 写 ODR, 或写 BSRR (Bit Set/Reset Register, 位设置/复位寄存器)
- 复用功能输出可由定时器、串口等外设接管
- 输出控制电路驱动 P-MOS 与 N-MOS
- 引脚实际电平仍可经 IDR 回读

输出模式的关键不是“写 0/1”，而是“在驱动哪只 MOS 管”。

图中寄存器名: BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

ODR 与 BSRR：一个保存状态，一个原子修改



ai14782

ODR: 输出锁存器

为什么会覆盖别的位？ODR 写回的是整组 16 位。

- ① 主程序先读 ODR: PA6=0、PA5=0
- ② 随后中断把 PA6 置 1
- ③ 主程序仍在旧副本上把 PA5 改为 1, 再把整组写回
- 结果: 旧副本中的 PA6=0 覆盖了中断刚写入的 PA6=1

```
tmp = GPIOA->ODR; /* 读旧值 */
tmp |= GPIO_ODR_ODR5;
GPIOA->ODR = tmp; /* 整组写回 */
```

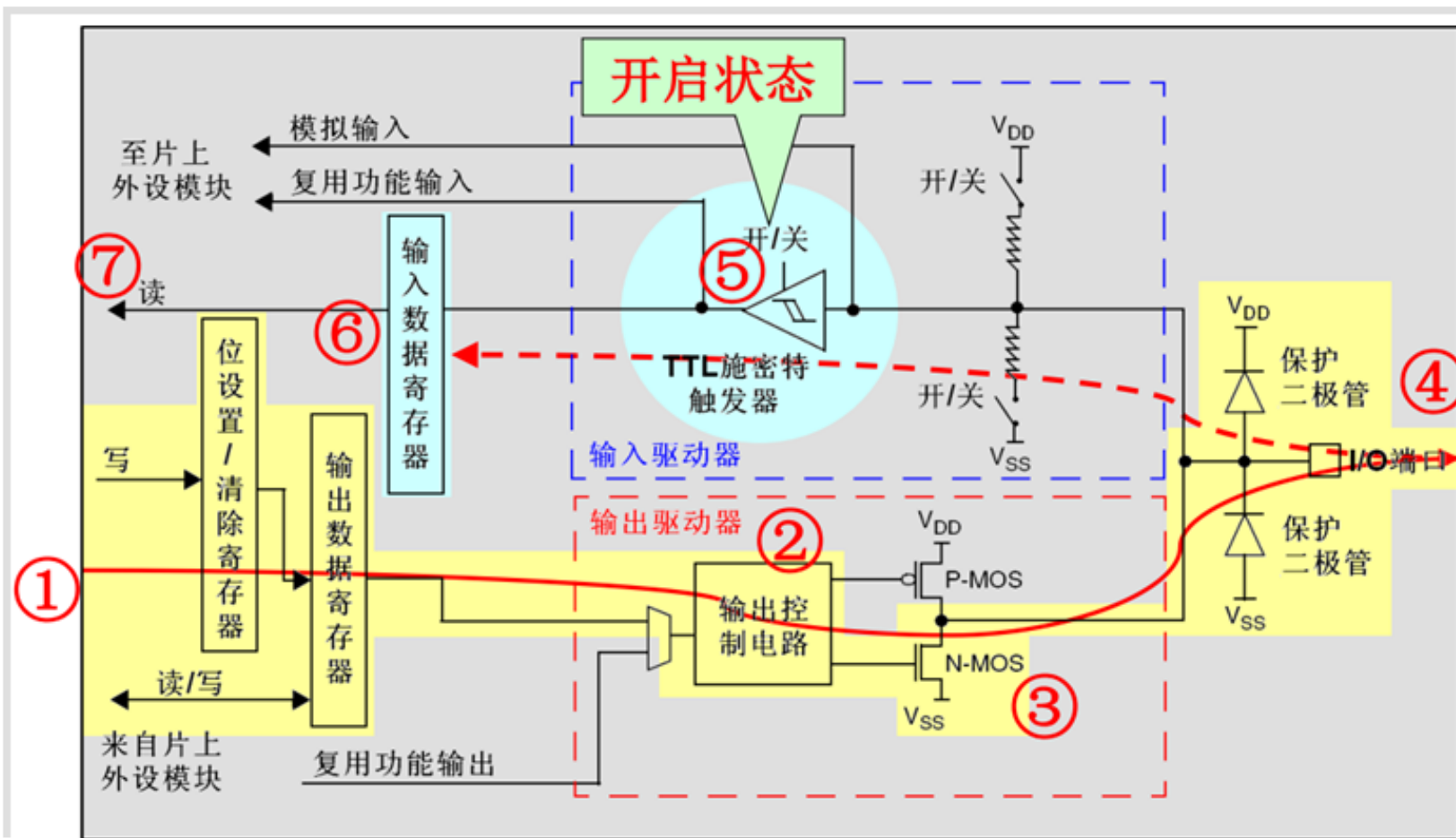
BSRR: 置位 / 复位命令

- BS5 写 1: 硬件只把 PA5 对应的 ODR 位设为 1
- 没有“先读旧值再整组写回”，所以 PA6 保持不变

```
GPIOA->BSRR = GPIO_BSRR_BS5;
```

ODR 的风险来自“整组旧副本写回”；BSRR 只提交目标位的置位/复位命令。

推挽输出：适合单一驱动的高速数字信号



- 编码：MODE>00, CNF=00
- 输出 1：P-MOS 导通，引脚被主动拉高
- 输出 0：N-MOS 导通，引脚被主动拉低
- 高、低电平均有较低输出阻抗，边沿通常更快
- 适合 LED、片选、时钟及点对点数字输出
- 共享线路必须保证任一时刻只有一个推挽驱动器工作

使用原则：一个输出驱动若干高阻输入；不要把两个持续使能的推挽输出直接并联。

推挽总线冲突：一端拉高，另一端拉低

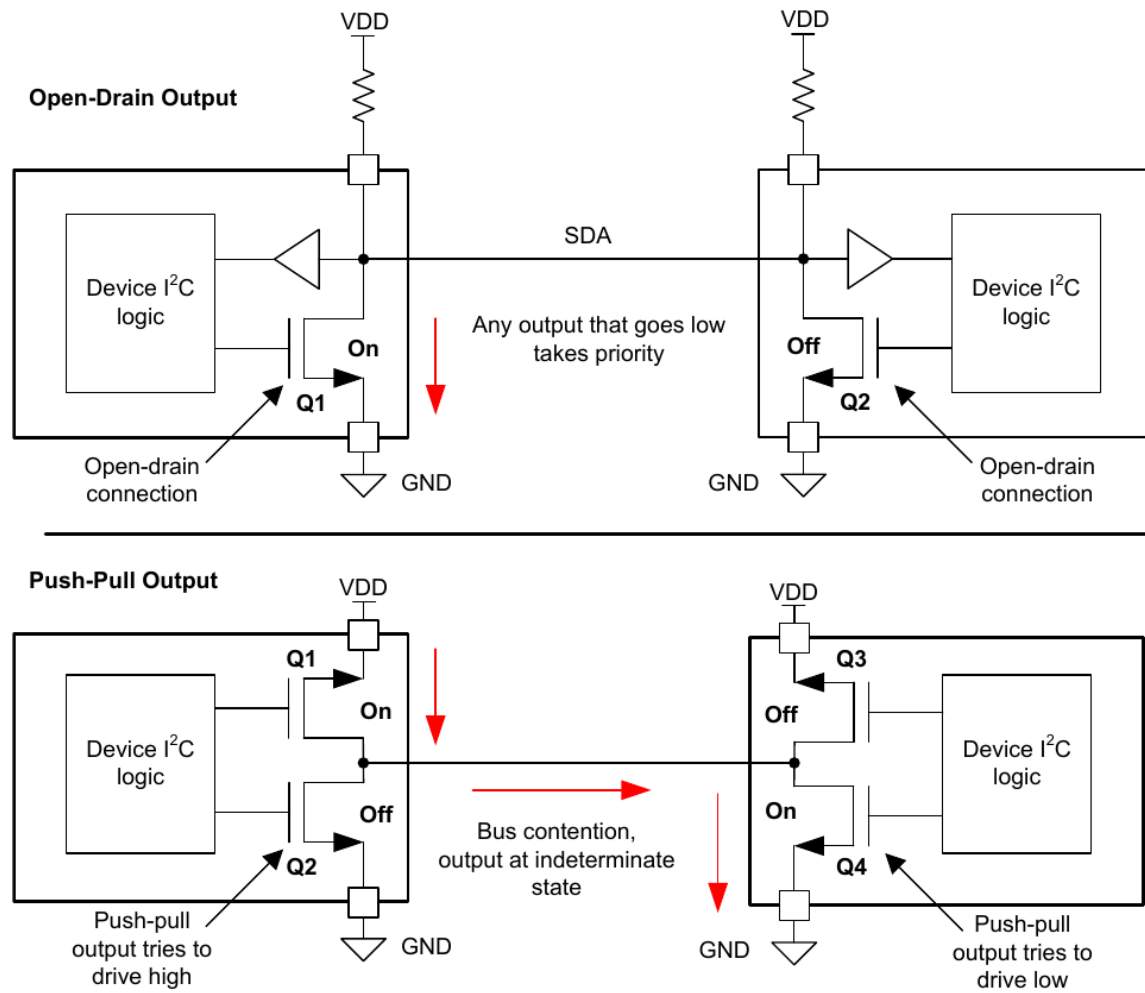


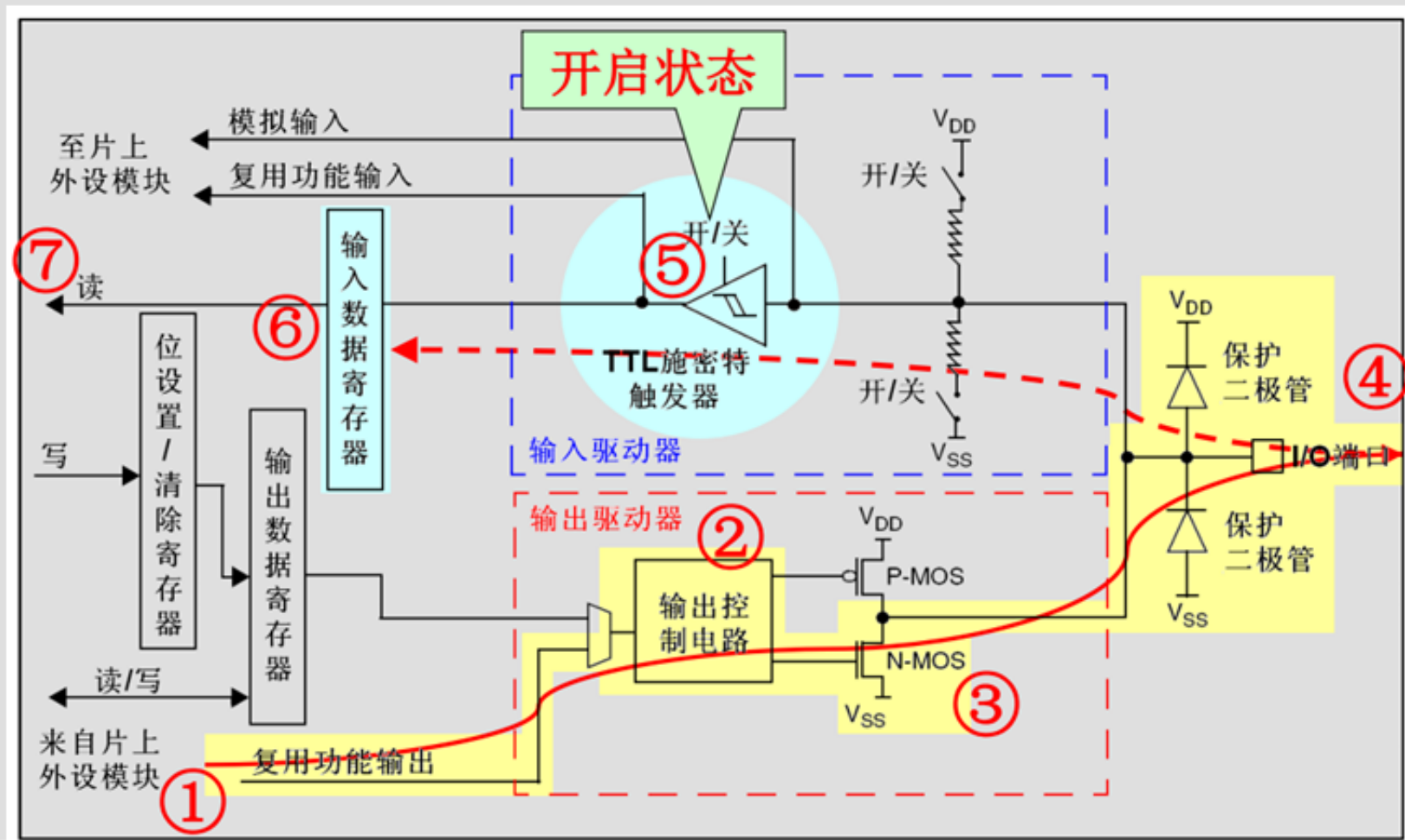
Figure 2-4. Comparison Between Open-Drain and Push-Pull Contention

冲突发生时

- 器件 A 输出 1，P-MOS 导通并向线路供电
- 器件 B 输出 0，N-MOS 导通并把线路拉向地
- 电流经 VDD、P-MOS、线路、N-MOS 流向 GND
- 通路阻抗很低，电流可能远超引脚允许值
- 线路电平也变得不确定，两个输出级都可能受损

推挽适合点对点或单一驱动。共享总线应使用三态控制，或改用开漏加上拉。

开漏输出： 适合共享线路与外部上拉

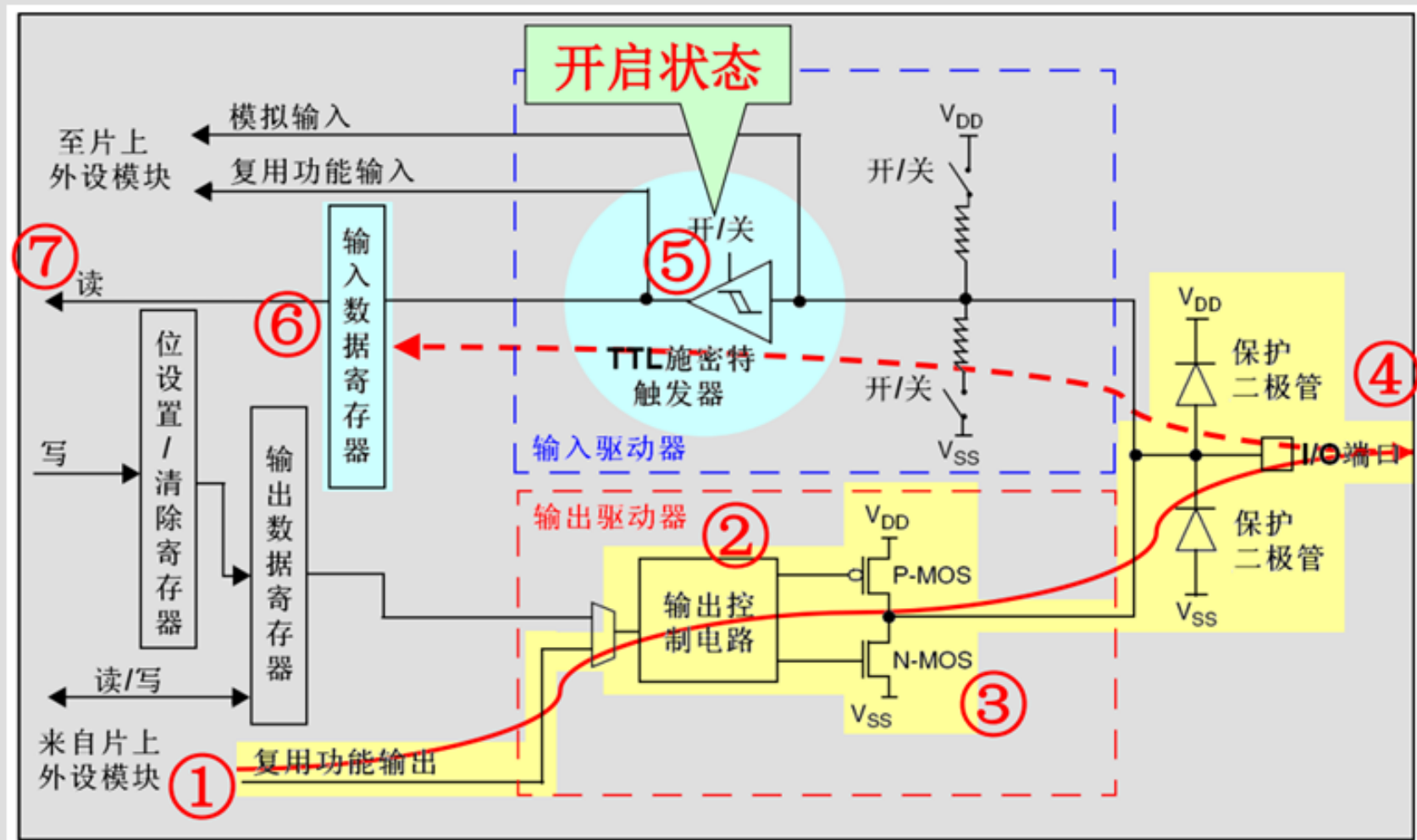


- 编码：MODE>00，CNF=01
- 输出 0：N-MOS 导通，引脚被拉低
- 输出 1：N-MOS 截止，引脚变为高阻态
- 高电平由外部上拉电阻与线路电容共同建立
- 适合 I²C、共享中断/故障线，以及满足耐压条件的电平转换
- 上升沿通常比推挽慢，上拉电阻需兼顾速度与功耗

线路上只允许器件“拉低”或“释放”，因此多个开漏输出可以安全共享

图中寄存器名：BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

复用推挽输出：片上外设成为 GPIO 数据源

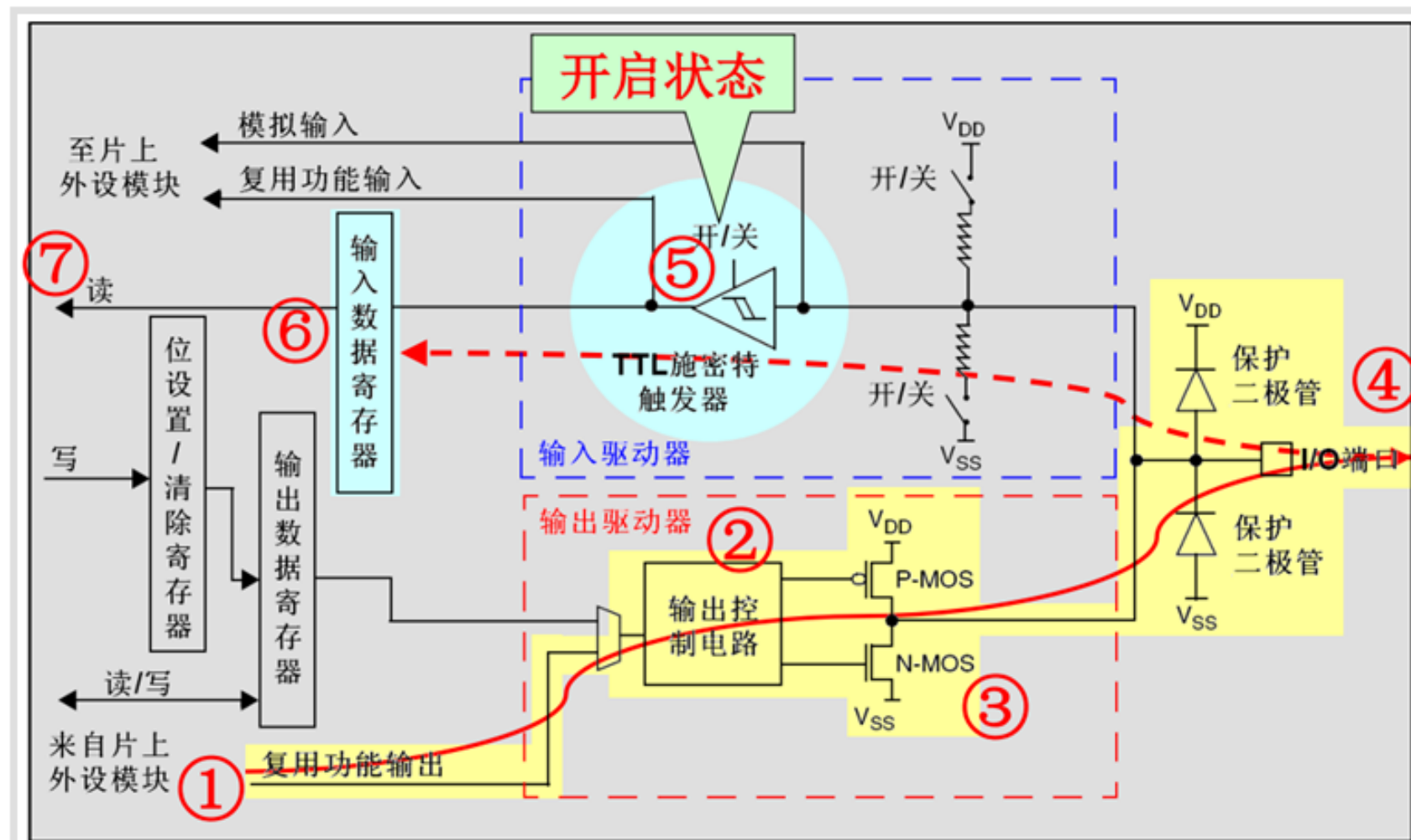


- 编码：MODE>00
CNF=10
- 输出选择器从 ODR 路径切换到片上外设输出
- CPU 只配置外设并写数据，外设硬件按时序驱动引脚
- P-MOS 与 N-MOS 仍负责实际的高、低电平驱动

GPIO 决定电气驱动方式，片上外设决定波形、时序和发送内容。

图中寄存器名：BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

复用开漏输出:



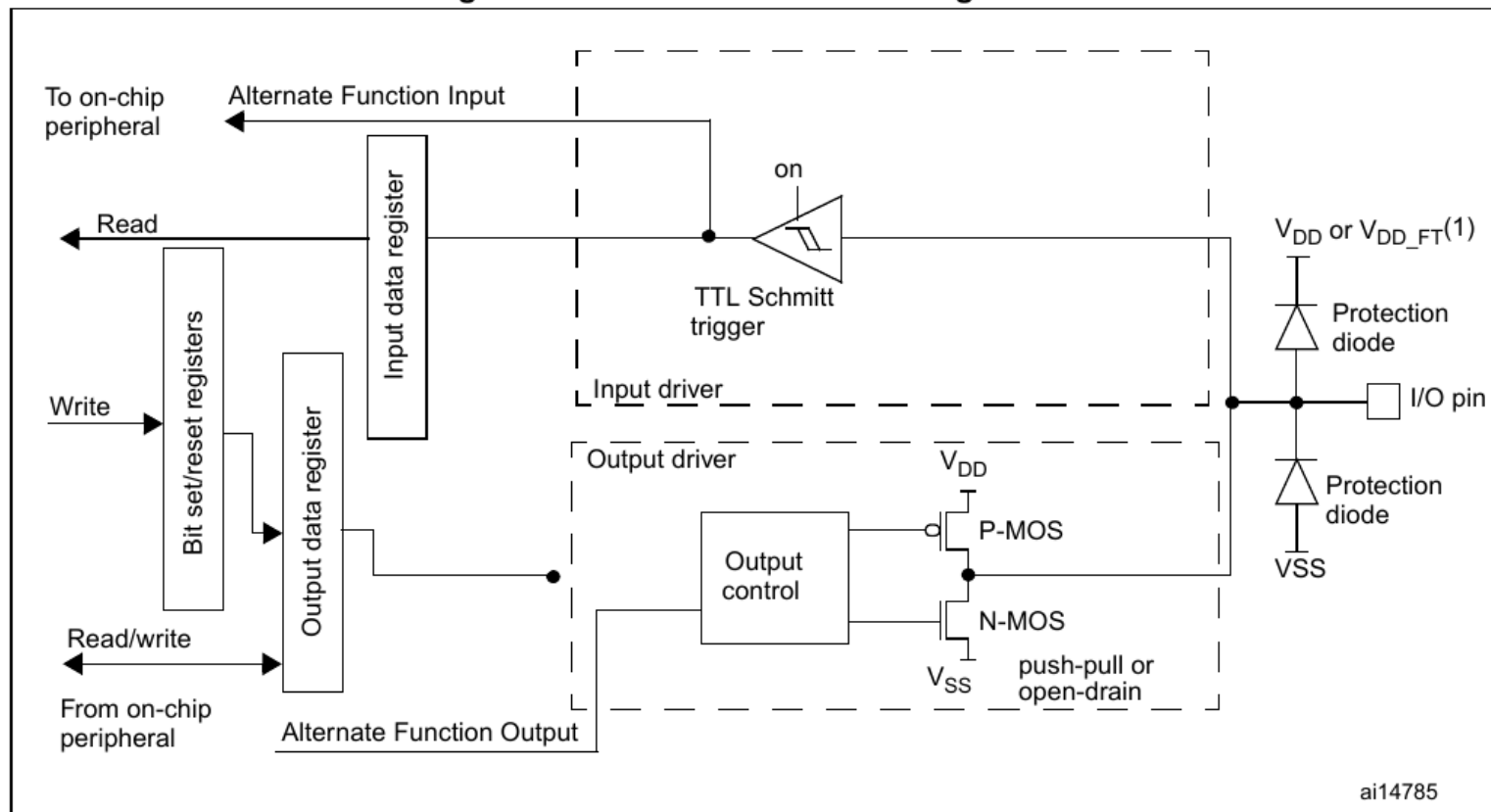
- 编码: MODE>00
CNF=11
- CPU 把地址和数据写入片上外设寄存器 (如IIC的寄存器), 不再逐位写 ODR
- 片上外设寄存器直接控制 N-MOS
- 外设发送 0 时拉低, 发送 1 时释放线路
- 外部上拉产生高电平, 并允许多个器件共享

通用开漏由 CPU 的 ODR 控制; 复用开漏由片上外设状态机控制。电气结构相同。

图中寄存器名: BSRR → 位设置/清除寄存器 ODR → 输出数据寄存器 IDR → 输入数据寄存器 CRL/CRH → 模式与速度配置

复用模式与片上外设

Figure 17. Alternate function configuration



1. V_{DD_FT} is a potential specific to 5-Volt tolerant I/Os, and different from V_{DD} .

复用输出

TIM、USART等功能产生的内部输出信号走复用输出通道

复用输入

引脚电平经过施密特触发器后，送入USART_RX、SPI_MISO 或定时器输入捕获电路。

STM32F1 没有单独的复用输入编码。RX、MISO 和输入捕获仍配置为输入，但信号会同时送到片上外设。

同一个 USART, TX 与 RX 的 GPIO 配置不同

USART1_TX: 复用输出

数据方向	USART1 发送移位寄存器送到引脚
配置编码	MODE > 00 CNF = 10: 复用推挽输出
常见用途	串口发送、PWM、SPI 时钟与 MOSI

USART1_RX: 输入路径接到外设

数据方向	引脚电平送入 USART1 接收器
配置编码	MODE = 00 CNF = 01: 浮空输入 或 CNF = 10: 上拉/下拉输入
常见用途	串口接收、SPI MISO 与定时器输入捕获

TX: USART 发送移位寄存器驱动 GPIO。RX: GPIO 输入电平送入 USART 接收器

引脚重映射：外设不变，连接到另一组引脚

Table 54. USART1 remapping

Alternate function	USART1_REMAP = 0	USART1_REMAP = 1
USART1_TX	PA9	PB6
USART1_RX	PA10	PB7

9.4.2 AF remap and debug I/O configuration register (AFIO_MAPR)

Address offset: 0x04

Reset value: 0x0000 0000

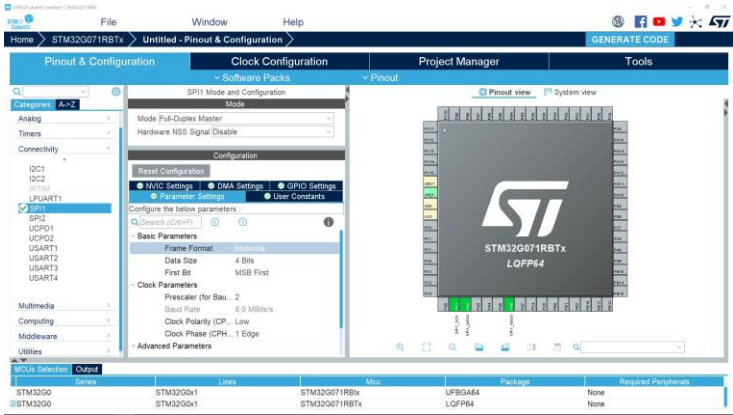
Memory map and bit definitions for low-, medium- high- and XL-density devices:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					SWJ_CFG[2:0]			Reserved			ADC2_E TRGRÉG _REMAP	ADC2_E TRGINJ _REMAP	ADC1_E TRGRÉG _REMAP	ADC1_E TRGINJ _REMAP	TIM5CH4 _IREMAP
					w	w	w				rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PD01_ REMAP	CAN_REMAP [1:0]		TIM4_ REMAP	TIM3_REMAP [1:0]	TIM2_REMAP [1:0]		TIM1_REMAP [1:0]		USART3_ REMAP[1:0]			USART2_ REMAP	USART1_ REMAP	I2C1_ REMAP	SPI1_ REMAP
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

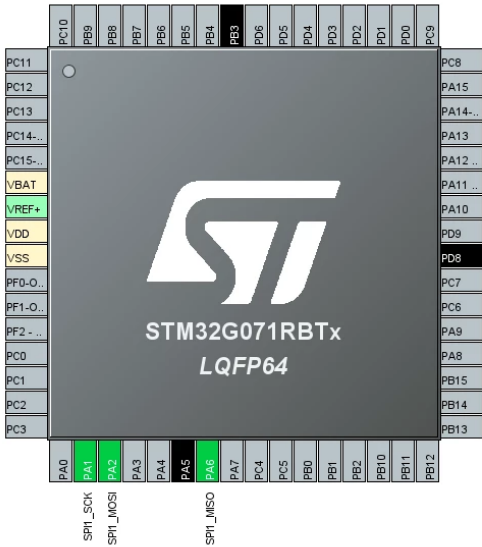
- AFIO = Alternate Function I/O，复用功能 I/O
- AFIO_MAPR 的 USART1_REMAP 位选择整组引脚
- 写 0：TX 使用 PA9，RX 使用 PA10
- 写 1：TX 改到 PB6，RX 改到 PB7
- 波特率、DMA、中断和 USART1 外设本身不变
- 访问 AFIO_MAPR 前先使能 AFIO 时钟

重映射改变“外设连到哪个引脚”，不会把 TX 变成输入，也不会把 RX 变成输出。

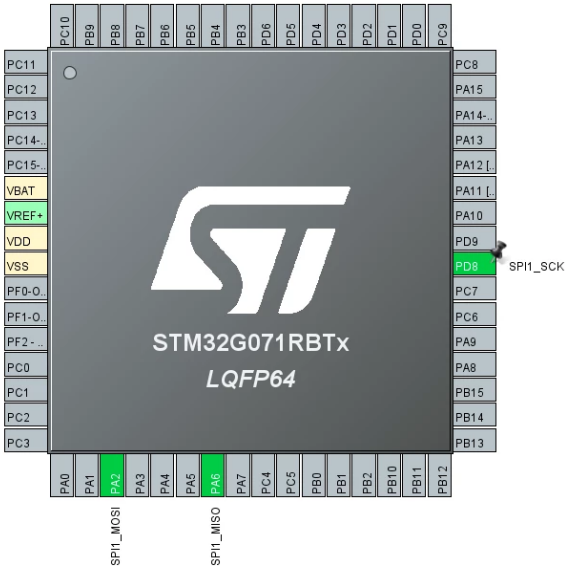
CubeMX：先看候选脚，再核对整组映射



① 使能外设并查看默认引脚



② Ctrl + 单击，显示可迁移引脚



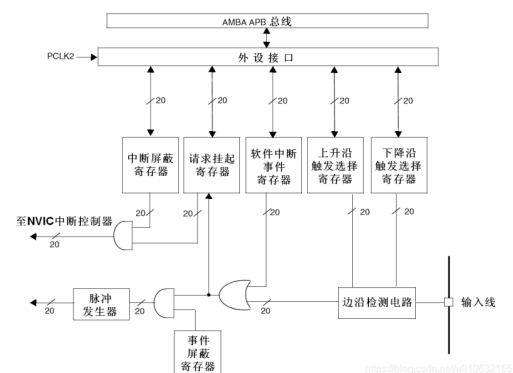
③ 拖到目标脚，检查其他信号

STM32F1 经常按“整组信号”重映射。移动 TX 后，应同时确认 RX、CK、CTS/RTS 等相关信号是否仍满足所选模式。

界面操作图来自 ST 官方通用示例；STM32F103 的合法组合以器件数据手册和 RM0008 为准。

外部中断 EXTI

刚才由 USART 状态位发出请求，现在改由 GPIO 边沿发出请求



轮询与中断：区别在 CPU 是否一直等

轮询

```
while (1) {  
    if (HAL_GPIO_ReadPin(KEY_GPIO_Port,  
KEY_Pin) == GPIO_PIN_RESET) {  
        key_pressed();  
    }  
}
```

轮询：CPU 主动反复读取

中断

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)  
{  
    if (pin == KEY_Pin) {  
        key_pressed();  
    }  
}
```

中断：边沿到来时硬件通知 CPU

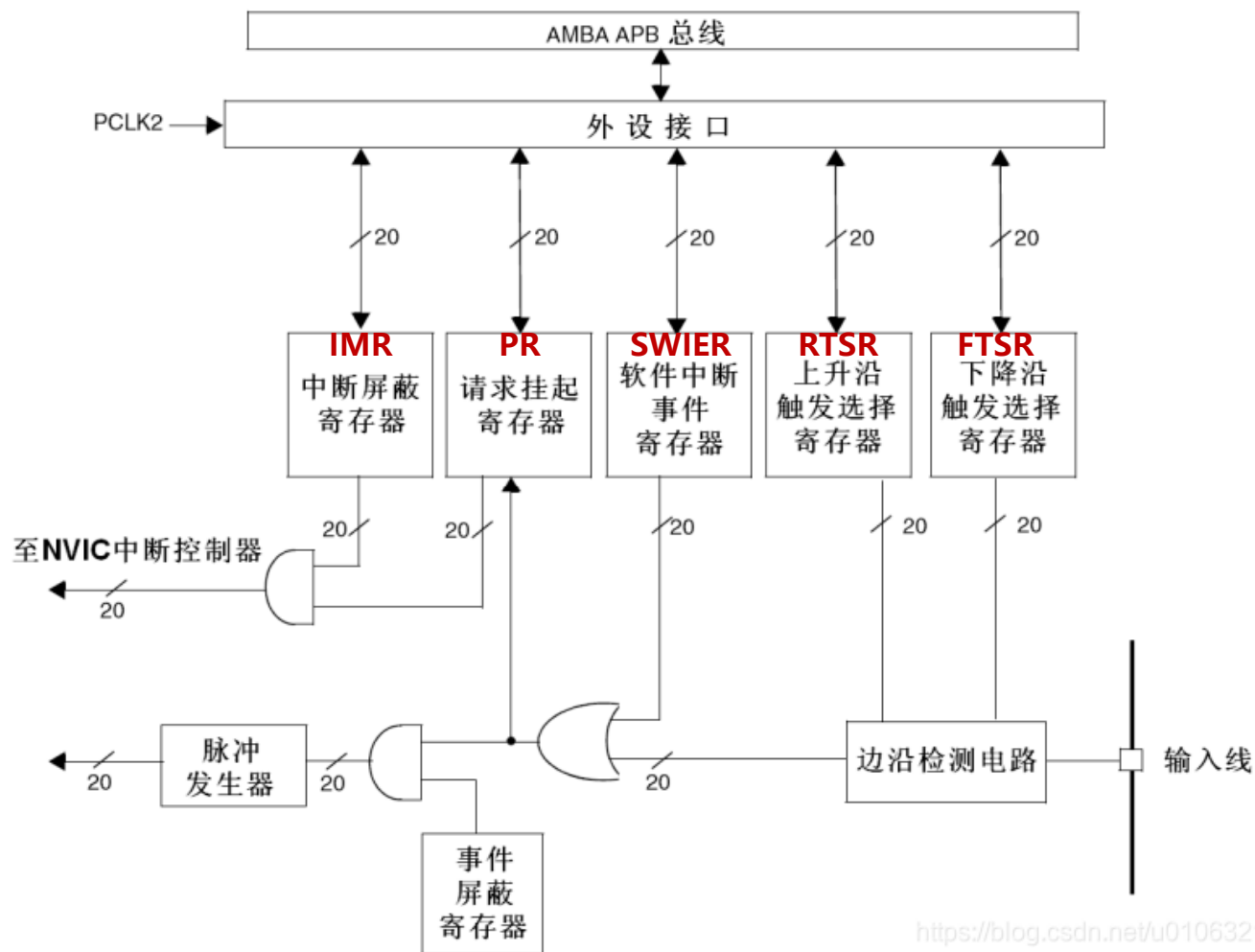
中断不是“更快的轮询”，而是改变了 CPU 与事件之间的协作方式。

一次 EXTI 中断要经过六个关口

1	GPIO 输入	施密特触发器把引脚电压整理为 0 或 1	输入电平
2	AFIO 映射	从 PAX、PBx、PCx 中选择一个，接到 EXTIx	AFIO_EXTICR
3	边沿选择	决定检测 0→1、1→0，或两种变化	RTSR / FTSR
4	记录发生	有效边沿把对应挂起位置 1	PR
5	允许送出	中断屏蔽位为 1，才把请求送往 NVIC	IMR
6	进入软件	NVIC 选择 IRQ，IRQHandler 调用 HAL，再进入回调	ISR / Callback

排错时按 1 到 6 顺序检查。前一级没有成立，后一级一定不会发生。

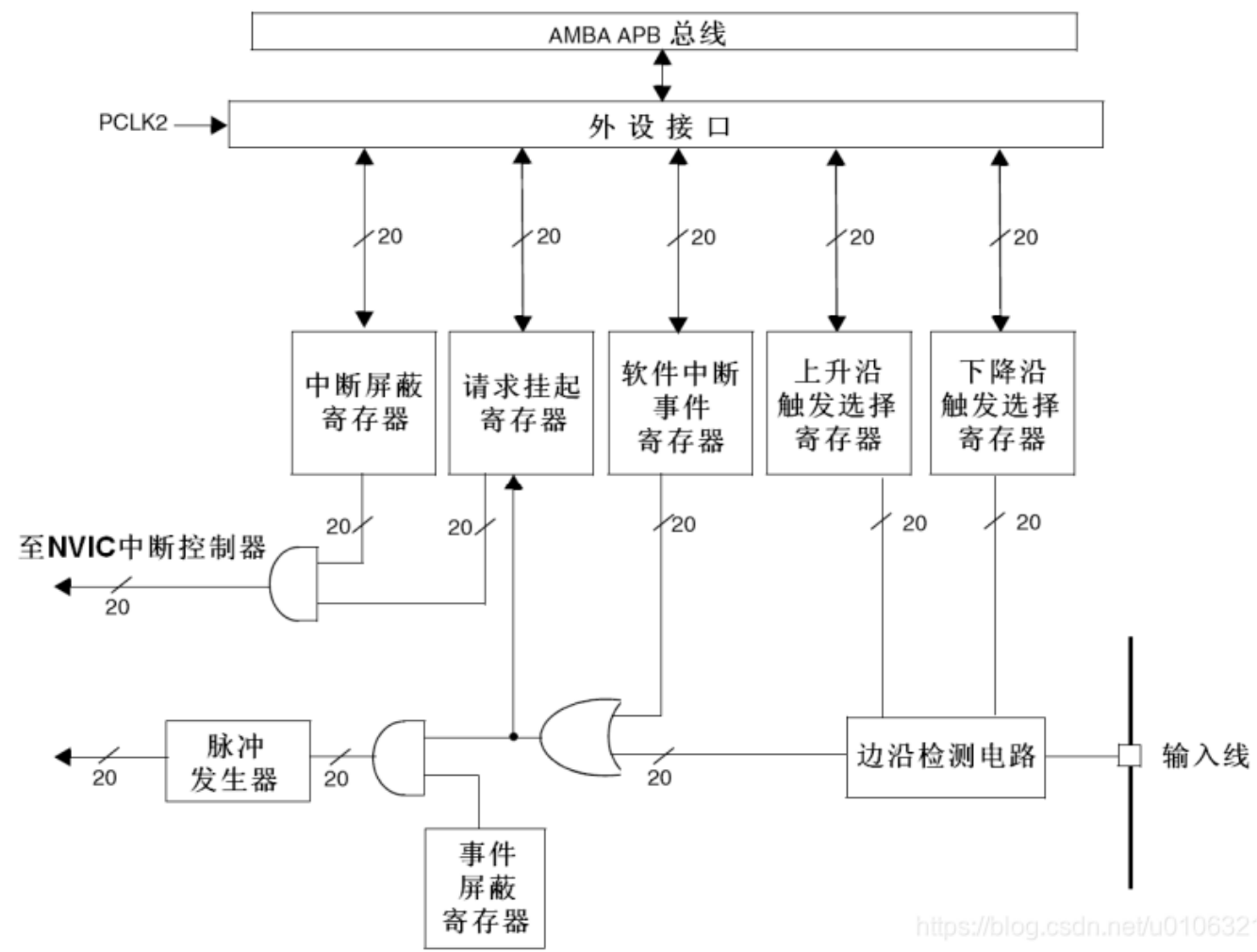
EXTI 总图：先认五个关键寄存器



- RTSR (Rising Trigger Selection Register) : 选择上升沿
- FTSR (Falling Trigger Selection Register) : 选择下降沿
- SWIER (Software Interrupt Event Register) : 软件触发
- PR (Pending Register) : 记录挂起请求
- IMR / EMR (Interrupt / Event Mask Register) : 控制两条输出

先产生触发，再形成挂起，最后经过屏蔽。

EXTI 有两条输出：事件与中断



中断请求

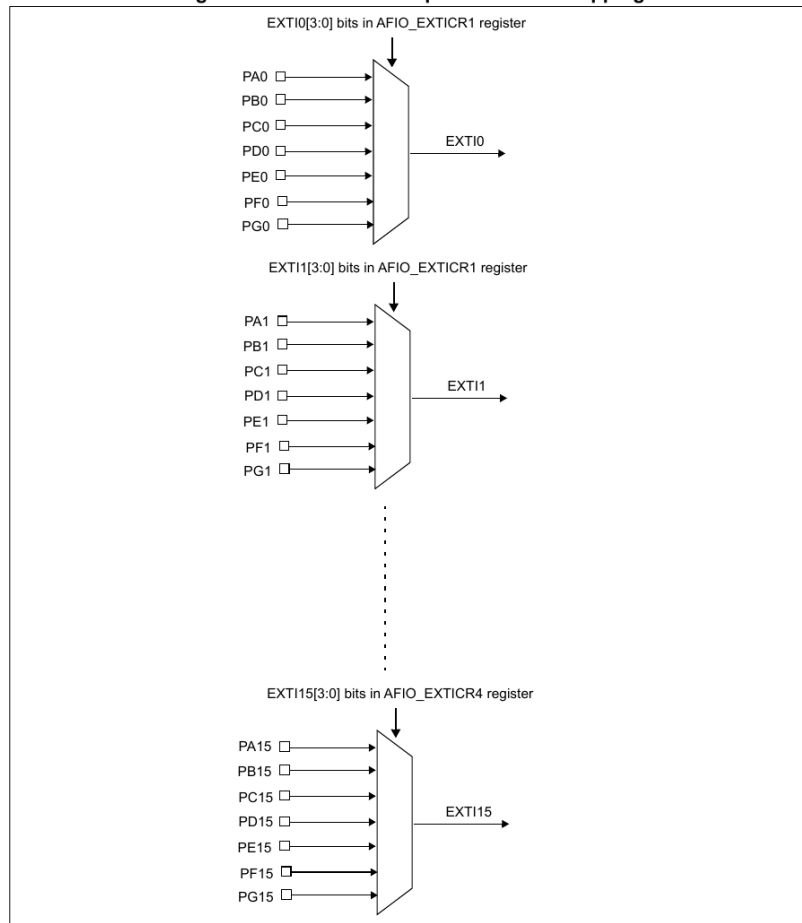
经过 IMR，送入 NVIC。CPU 可暂停当前程序并执行 ISR。

事件脉冲

经过 EMR，可触发硬件事件或唤醒等机制，不一定进入 ISR。

GPIO 先映射到 EXTI 线，再谈触发

Figure 21. External interrupt/event GPIO mapping



1. To configure the AFIO_EXTICRx for the mapping of external interrupt/event lines onto GPIOs, the AFIO clock should first be enabled. Refer to [Section 7.3.7: APB2 peripheral clock enable register \(RCC_APB2ENR\)](#) for low-, medium-, high- and XL-density devices and, to [Section 8.3.7: APB2 peripheral clock enable register \(RCC_APB2ENR\)](#) for connectivity line devices.

- EXTI0 只能从 PA0、PB0、PC0.....中选择一个端口来源
- EXTI1 同理，只接受 x1 引脚
- 映射由 AFIO_EXTICR1 ~ 4 完成
- 配置映射前要先使能 AFIO 时钟

EXTI 线号由引脚编号决定；端口字母由 AFIO 选择。

AFIO_EXTICR: 线号固定，端口来源可选

引脚编号	对应 EXTI 线	AFIO 配置寄存器
0 ~ 3	EXTI0 ~ EXTI3	AFIO_EXTICR1
4 ~ 7	EXTI4 ~ EXTI7	AFIO_EXTICR2
8 ~ 11	EXTI8 ~ EXTI11	AFIO_EXTICR3
12 ~ 15	EXTI12 ~ EXTI15	AFIO_EXTICR4

例：PB7 → EXTI7

- ① 脚号是 7，所以只能进入 EXTI7
- ② EXTI7 位于 AFIO_EXTICR2 的 [15:12]
- ③ 端口 B 的编码为 0001
- ④ 最终选择 PB7，PA7、PC7 等来源断开

```
GPIO_InitStruct.Pin = GPIO_PIN_7;
GPIO_InitStruct.Mode =
GPIO_MODE_IT_FALLING;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

HAL_GPIO_Init() 根据 GPIOB 与 GPIO_PIN_7，把 EXTICR2 的 EXTI7 字段写成端口 B。

同一条 EXTI7 线一次只能连接一个 x7 引脚。

边沿检测：比较变化前后的两个电平

变化前	变化后	发生了什么	需要使能
0	0	电平未变	无
0	1	上升沿	RTSRx = 1
1	0	下降沿	FTSRx = 1
1	1	电平未变	无

上拉按键的四个时刻

空闲	1	没有边沿
按下瞬间	1 → 0	下降沿，FTSR 可触发
持续按住	0	没有新边沿
松开瞬间	0 → 1	上升沿，RTSR 可触发

边沿是“变化的瞬间”，不是保持的电平。RTSR 与 FTSR 同时为 1 时，按下和松开都会触发。

边沿选择：RTSR 看上升，FTSR 看下降

9.3.3 上升沿触发选择寄存器(EXTI_RTSR)

偏移地址：0x08

复位值：0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留												TR19	TR18	TR17	TR16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
位31:19		保留，必须始终保持为复位状态(0)。													
位18:0		TRx : 线x上的上升沿触发事件配置位 (Rising trigger event configuration bit of line x) 0: 禁止输入线x上的上升沿触发(中断和事件) 1: 允许输入线x上的上升沿触发(中断和事件) 注：位19只适用于互联型产品，对于其它产品为保留位。													

注意: 外部唤醒线是边沿触发的，这些线上不能出现毛刺信号。
在写EXTI_RTSR寄存器时，在外部中断线上的上升沿信号不能被识别，挂起位也不会被置位。
在同一中断线上，可以同时设置上升沿和下降沿触发。即任一边沿都可触发中断。

ST中文论坛

RTSRx=1：检测 0→1

9.3.4 下降沿触发选择寄存器(EXTI_FTSR)

偏移地址：0x0C

复位值：0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留												TR19	TR18	TR17	TR16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
位31:19		保留，必须始终保持为复位状态(0)。													
位18:0		TRx : 线x上的下降沿触发事件配置位 (Falling trigger event configuration bit of line x) 0: 禁止输入线x上的下降沿触发(中断和事件) 1: 允许输入线x上的下降沿触发(中断和事件) 注：位19只适用于互联型产品，对于其它产品为保留位。													

注意: 外部唤醒线是边沿触发的，这些线上不能出现毛刺信号。
在写EXTI_FTSR寄存器时，在外部中断线上的下降沿信号不能被识别，挂起位不会被置位。
在同一中断线上，可以同时设置上升沿和下降沿触发。即任一边沿都可触发中断。

ST中文论坛

FTSRx=1：检测 1→0

SWIER: 不用按键，也能制造一次 EXTI 请求

9.3.5 软件中断事件寄存器(EXTI_SWIER)

偏移地址: 0x10

复位值: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留												SWIER 19	SWIER 18	SWIER 17	SWIER 16
												rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWIER 15	SWIER 14	SWIER 13	SWIER 12	SWIER 11	SWIER 10	SWIER 9	SWIER 8	SWIER 7	SWIER 6	SWIER 5	SWIER 4	SWIER 3	SWIER 2	SWIER 1	SWIER 0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

位31:19	保留，必须始终保持为复位状态(0)。
位18:0	SWIERx : 线x上的软件中断 (Software interrupt on line x) 当该位为'0'时，写'1'将设置EXTI_PR中相应的挂起位。如果在EXTI_IMR和EXTI_EMR中允许产生该中断，则此时将产生一个中断。 注：通过清除EXTI_PR的对应位(写入'1')，可以清除该位为'0'。 注：位19只适用于互联型产品，对于其它产品为保留位。



- 对 SWIERx 写 1，可产生与外部边沿相似的触发
- 它绕过了 GPIO 引脚、电气接法和边沿检测
- 如果软件触发可进 ISR，而按键不行，优先检查 GPIO、AFIO 映射和 RTSR/FTSR
- 如果软件触发也不行，再检查 IMR、NVIC、向量表和 ISR 名称

SWIER 是分段排错工具。

PR 记录发生过，IMR 决定能否送出

9.3.6 挂起寄存器(EXTI_PR)

偏移地址: 0x14
复位值: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留												PR19	PR18	PR17	PR16
												rc	wl	rc	wl
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc	wl	rc	wl	rc	wl	rc	wl	rc	wl	rc	wl	rc	wl	rc	wl
位31:19		保留, 必须始终保持为复位状态(0)。													
位18:0		PRx : 挂起位 (Pending bit) 0: 没有发生触发请求 1: 发生了选择的触发请求 当在外部中断线上发生了选择的边沿事件, 该位被置'1'。在该位中写入'1'可以清除它, 也可以通过改变边沿检测的极性清除。 注: 位19只适用于互联型产品, 对于其它产品为保留位。													

PRx=1: 该线已经触发; 写 1 清除

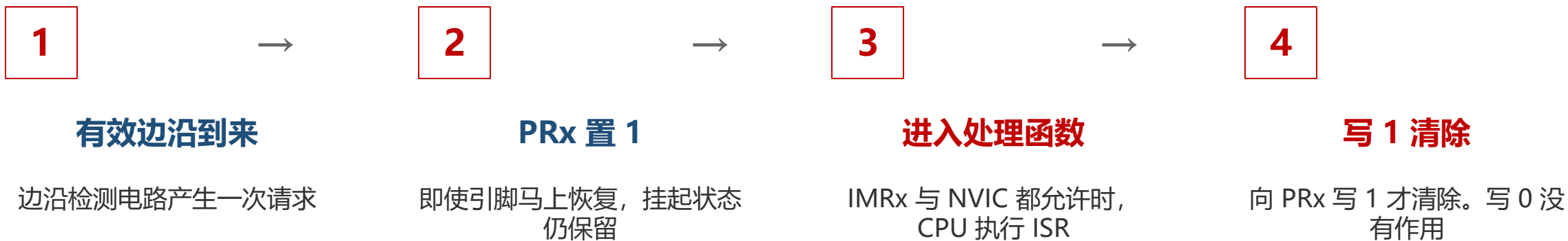
9.3.1 中断屏蔽寄存器(EXTI_IMR)

偏移地址: 0x00
复位值: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留												MR19	MR18	MR17	MR16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR4	MR4	MR3	MR2	MR1	MRO
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
位31:20		保留, 必须始终保持为复位状态(0)。													
位19:0		MRx : 线x上的中断屏蔽 (Interrupt Mask on line x) 0: 屏蔽来自线x上的中断请求; 1: 开放来自线x上的中断请求。 注: 位19只适用于互联型产品, 对于其它产品为保留位。													

IMRx=1: 允许中断请求继续前往 NVIC

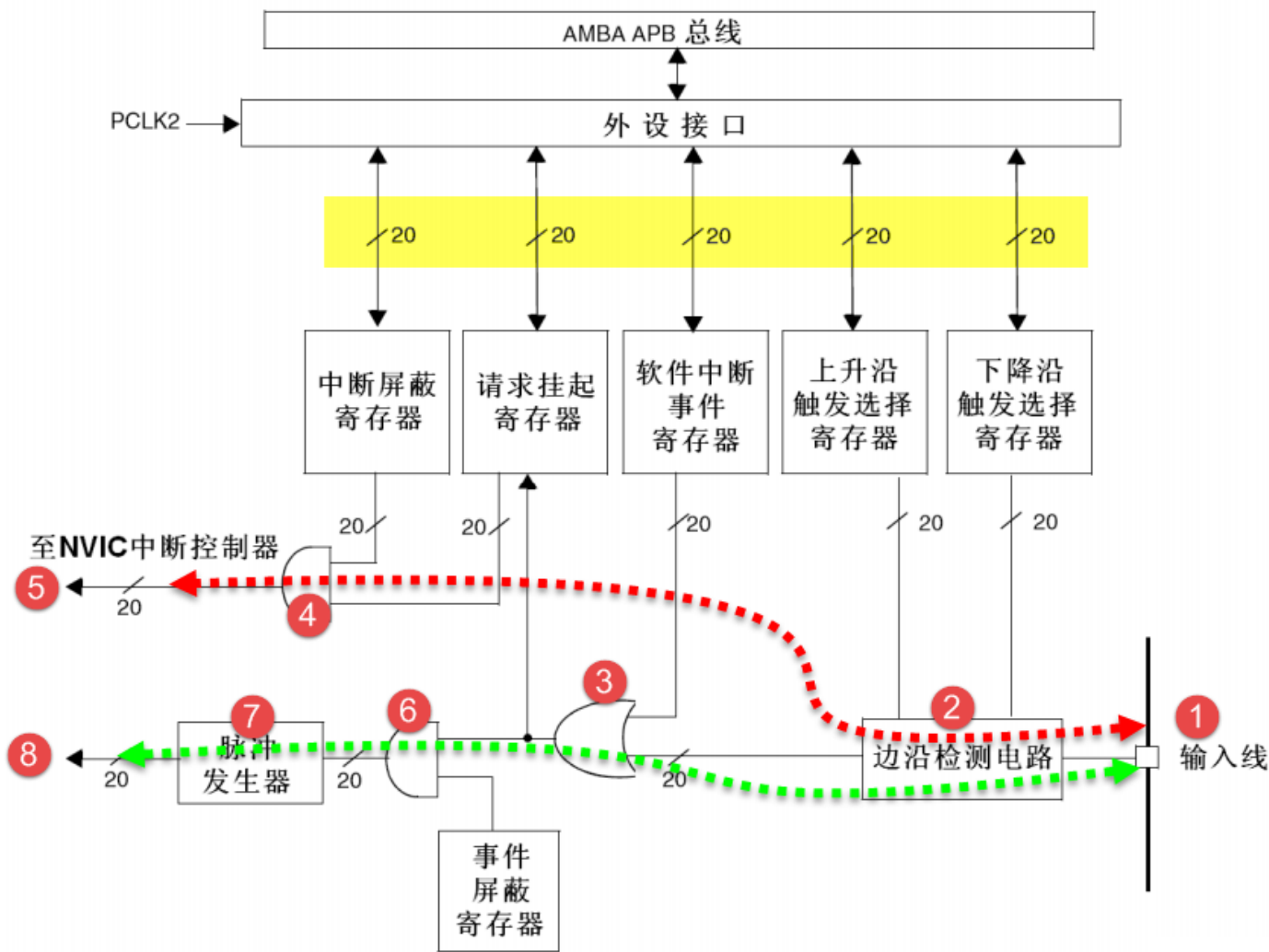
PR：挂起位把瞬间边沿保存下来



```
if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_7) != RESET)
{
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_7); // EXTI->PR = GPIO_PIN_7
    HAL_GPIO_EXTI_Callback(GPIO_PIN_7);
}
```

PR 是“发生过”的记忆。IMR 只控制是否向 NVIC 放行, 不会替你清除 PR。

一次外部中断，在 EXTI 内部怎样走？



① 输入线

② 边沿检测

③ 形成挂起

④ 经过 IMR

⑤ 请求 NVIC

任何一步没有配置，链路都会在该处中断。

NVIC：嵌套向量中断控制器

NVIC = Nested Vectored Interrupt Controller

Nested (嵌套)

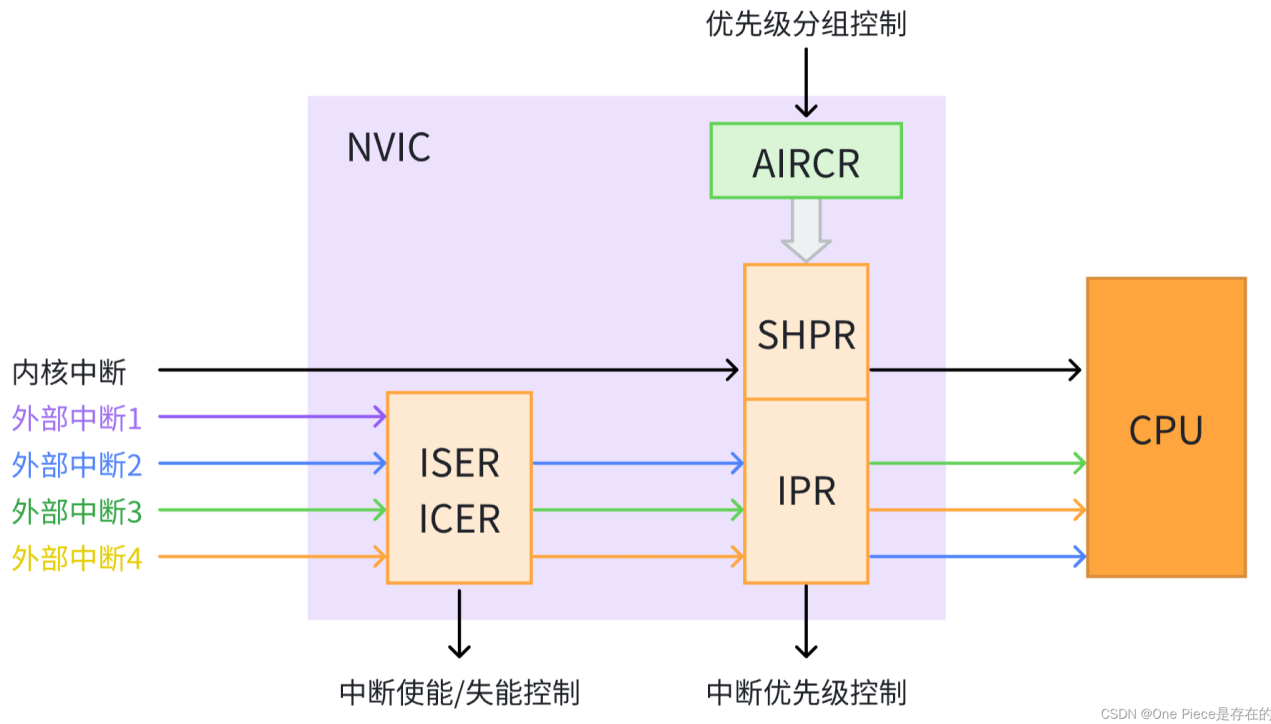
高优先级中断可以抢占正在执行的低优先级中断。

Vectorized (向量)

硬件根据中断号找到对应的处理函数入口。

Controller (控制器)

管理使能、挂起和优先级，再把选中的请求交给 CPU。



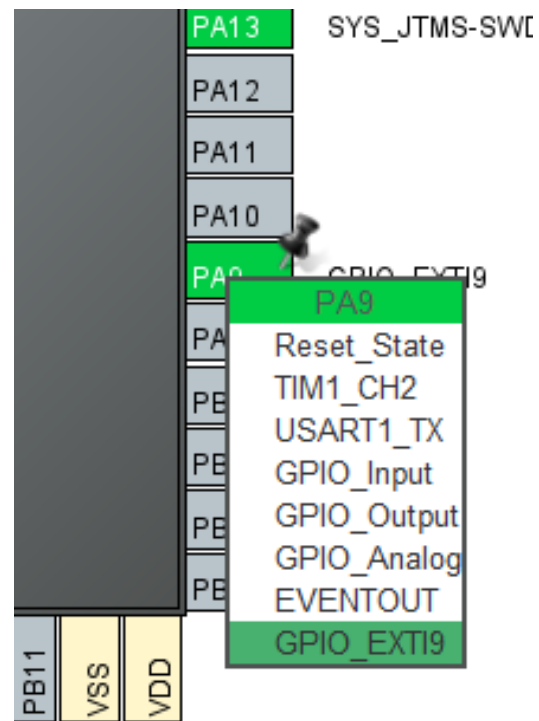
IRQ = Interrupt Request (中断请求)

ISR = Interrupt Service Routine (中断服务程序)

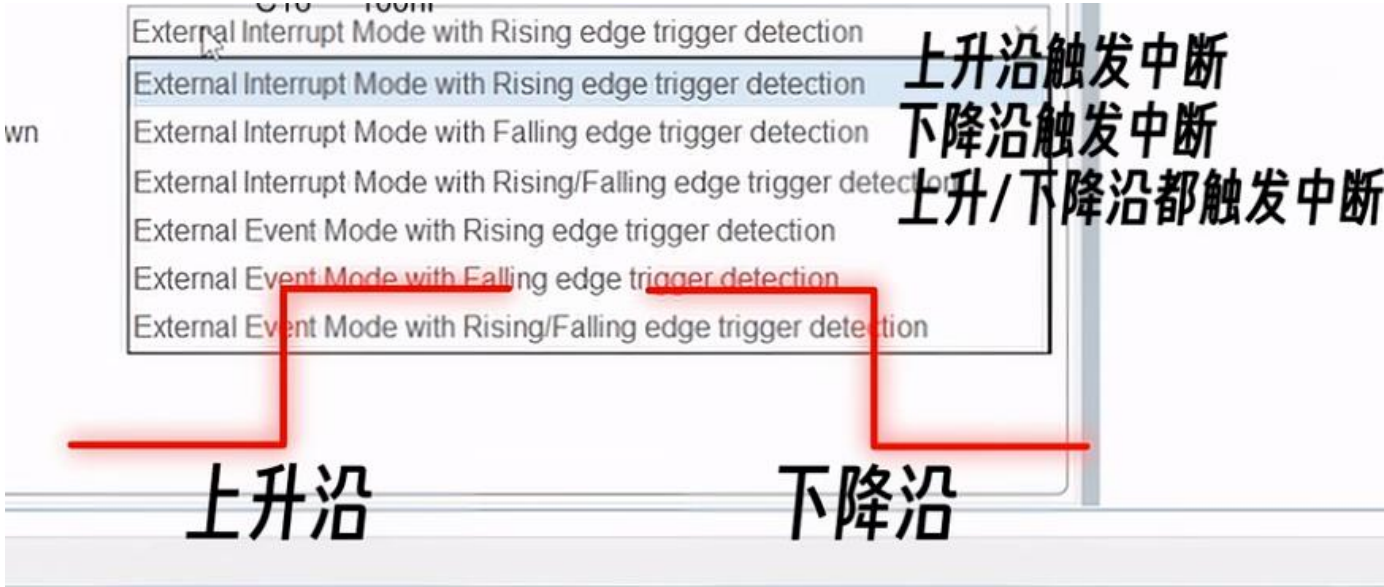
图中 ISER/ICER 管使能, IPR 管优先级, AIRCR 管优先级分组。

EXTI 产生请求, NVIC 安排请求, CPU 执行 ISR。

CubeMX: 先配置 EXTI 引脚和触发边沿



① 在 Pinout 中把 你想要设置的引脚添加外部中断



② 在 GPIO Mode 中选择上升沿、下降沿或双边沿，并设置上拉/下拉

CubeMX: 在 NVIC 中使能通道并设置优先级

SYS	Hard fault interrupt	☒	0	0
WWDG	Memory management fault	☒	0	0
	Prefetch fault, memory access fault	☒	0	0
	Undefined instruction or illegal state	☒	0	0
Analog >	System service call via SWI instruction	☒	0	0
	Debug monitor	☒	0	0
Timers >	Pendable request for system service	☒	0	0
	Time base: System tick timer	☒	15	0
Connectivity >	PVD interrupt through EXTI line 16	☐	0	0
	Flash global interrupt	☐	0	0
Computing >	RCC global interrupt	☐	0	0
Middleware >	EXTI line[15:10] interrupts	☒	0	0

1 打开 System Core → NVIC

2 找到对应 EXTI IRQ 并勾选 Enabled

3 设置 Priority Group

4 设置 Preemption Priority 与 Sub Priority

Configuration			
Group By Peripherals			
☒ GPIO	☒ RCC	☒ SYS	☒ NVIC
NVIC Interrupt Table		Add	Preemption Priority
EXTI line[9:5] interrupts		☒	0

CubeMX 通常生成:
HAL_NVIC_SetPriority(...)
HAL_NVIC_EnableIRQ(...)

EXTI 线与 IRQ：有的独享，有的共享

中断/事件线	输入源
EXTI0	PX0 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI1	PX1 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI2	PX2 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI3	PX3 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI4	PX4 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI5	PX5 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI6	PX6 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI7	PX7 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI8	PX8 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI9	PX9 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI10	PX10 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI11	PX11 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI12	PX12 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI13	PX13 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI14	PX14 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI15	PX15 (X 可为 A, B, C, D, E, F, G, H, I)
EXTI16	PVD 输出
EXTI17	RTC 闹钟事件
EXTI18	USB 唤醒事件
EXTI19	以太网唤醒事件（只适用互联型）

- EXTI0 ~ 4：每条线各有一个 IRQ
- EXTI5 ~ 9：共享 EXTI9_5_IRQn
- EXTI10 ~ 15：共享 EXTI15_10_IRQn
- 共享 ISR 中必须检查具体是哪一条线挂起
- HAL 会根据 GPIO_PIN_x 分发回调

EXTI7 的入口是 EXTI9_5_IRQHandler。

共享 IRQ：一个入口服务多条 EXTI 线

stm32f1xx_it.c

```
void EXTI9_5_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_6);
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_8);
}
```

EXTI5 ~ 9 共用 EXTI9_5_IRQHandler

每次 HAL 调用只检查一个 pin

- 只有 EXTI6 挂起时，PIN_6 分支清 PR6 并进入回调；PIN_8 分支立即返回
- EXTI6 与 EXTI8 同时挂起时，两次检查都成立，因此同一次 IRQ 入口会调用两次回调
- 回调参数是 GPIO_PIN_6 或 GPIO_PIN_8。应用代码必须先判断 pin
- 共享的是 NVIC 入口，PR6 与 PR8 仍是互相独立的挂起位

“共享 IRQ”不代表请求混在一起。HAL 通过逐位检查 PR，把它们重新分开。

优先级：数值越小，逻辑优先级越高

抢占优先级

决定一个中断能否打断另一个正在执行的中断。

响应优先级（子优先级）

当多个同级请求同时等待时，决定先进入哪一个。

优先级数值：0 比 1 高，1 比 2 高

- 优先级分组决定多少位用于抢占、多少位用于子优先级
- 同一项目应统一分组，不要在不同模块重复改分组
- ISR 越短，越少依赖复杂的优先级技巧

HAL 中断代码：入口、分发、回调

stm32f1xx_it.c

```
void EXTI0_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
}
```

IRQHandler: 硬件入口

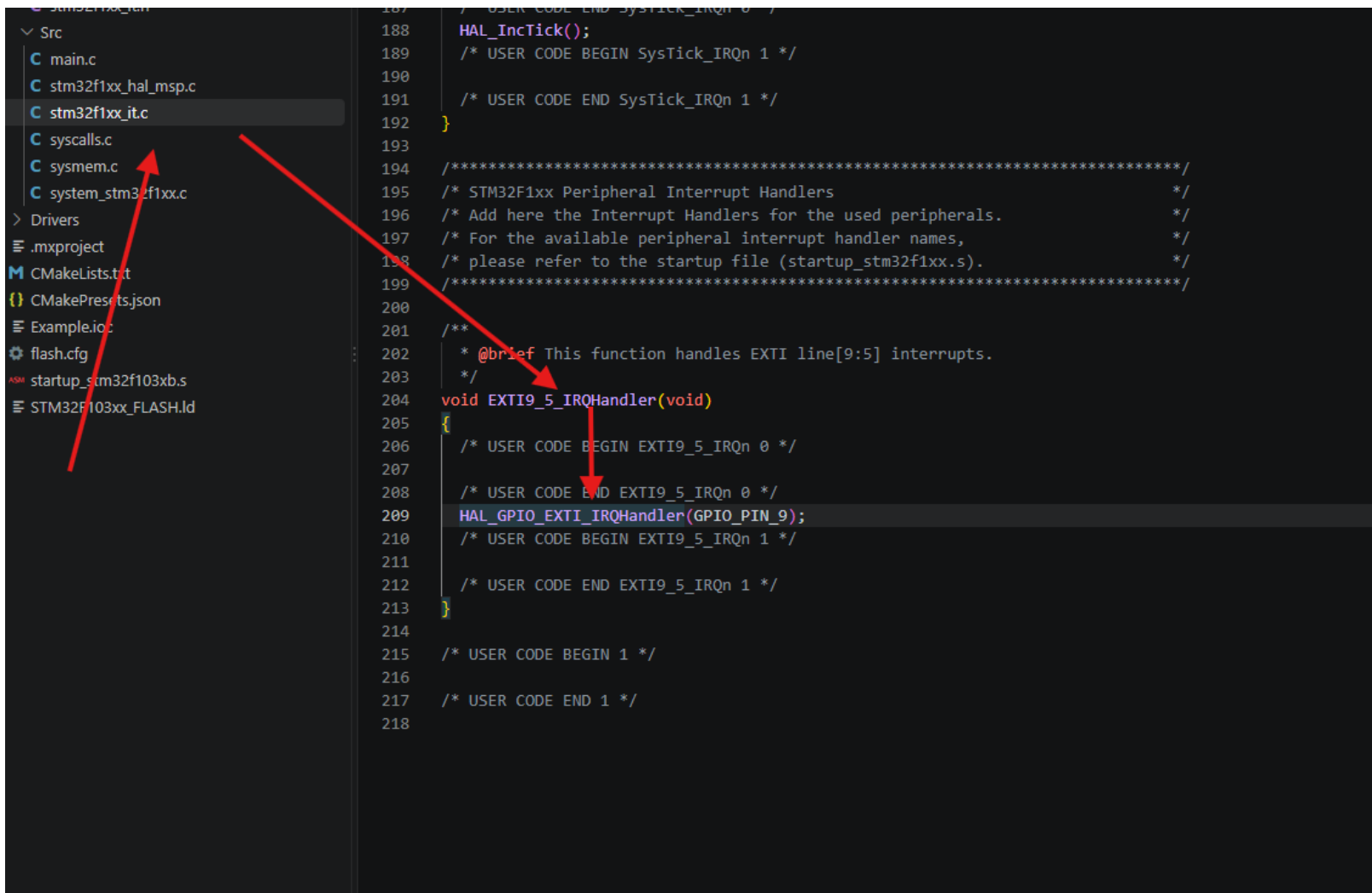
main.c / 用户代码文件

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
    if (pin == GPIO_PIN_0) {
        HAL_GPIO_TogglePin(
            LED_GPIO_Port,
            LED_Pin);
    }
}
```

Callback: 应用逻辑

- 不要删除 HAL_GPIO_EXTI_IRQHandler()
- 共享 IRQ 时, HAL 根据 pin 继续分发
- 回调中先判断 pin, 再执行短小动作

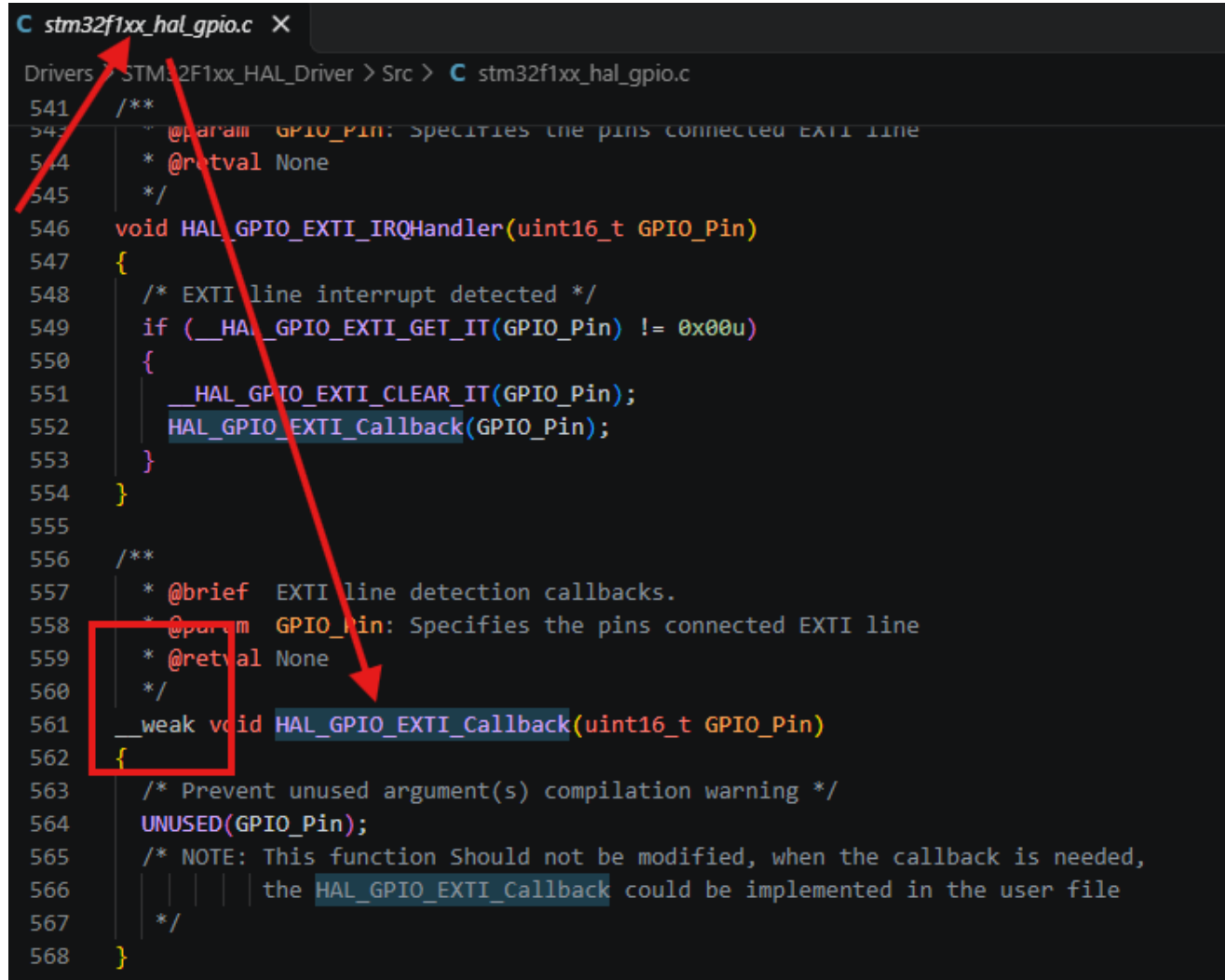
HAL 中断代码：入口、分发、回调



```
187  /* USER CODE END SysTick_IRQn 0 */
188  HAL_IncTick();
189  /* USER CODE BEGIN SysTick_IRQn 1 */
190
191  /* USER CODE END SysTick_IRQn 1 */
192  }
193
194  /*****
195   * STM32F1xx Peripheral Interrupt Handlers
196   * Add here the Interrupt Handlers for the used peripherals.
197   * For the available peripheral interrupt handler names,
198   * please refer to the startup file (startup_stm32f1xx.s).
199   *****/
200
201  /**
202   * @brief This function handles EXTI line[9:5] interrupts.
203   */
204  void EXTI9_5_IRQHandler(void)
205  {
206      /* USER CODE BEGIN EXTI9_5_IRQn 0 */
207
208      /* USER CODE END EXTI9_5_IRQn 0 */
209      HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_9);
210      /* USER CODE BEGIN EXTI9_5_IRQn 1 */
211
212      /* USER CODE END EXTI9_5_IRQn 1 */
213  }
214
215  /* USER CODE BEGIN 1 */
216
217  /* USER CODE END 1 */
218
```

HAL_GPIO_EXTI_IRQHandler: 检查、清除、回调

HAL 的默认 Callback 是 `__weak`。用户实现同名函数后，链接器会采用用户版本。



```
C stm32f1xx_hal_gpio.c X
Drivers > STM32F1xx_HAL_Driver > Src > C stm32f1xx_hal_gpio.c
541 /**
542  * @param GPIO_Pin: Specifies the pins connected EXTI line
543  * @retval None
544  */
545 void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
546 {
547     /* EXTI line interrupt detected */
548     if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
549     {
550         __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
551         HAL_GPIO_EXTI_Callback(GPIO_Pin);
552     }
553 }
554
555 /**
556  * @brief EXTI line detection callbacks.
557  * @param GPIO_Pin: Specifies the pins connected EXTI line
558  * @retval None
559  */
560 __weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
561 {
562     /* Prevent unused argument(s) compilation warning */
563     UNUSED(GPIO_Pin);
564     /* NOTE: This function Should not be modified, when the callback is needed,
565      the HAL_GPIO_EXTI_Callback could be implemented in the user file
566      */
567 }
568 }
```

HAL_GPIO_EXTI_IRQHandler: 检查、清除、回调

HAL 库中的核心代码

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != RESET)
    {
        __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
        HAL_GPIO_EXTI_Callback(GPIO_Pin);
    }
}
```

1

检查 PR

GET_IT 读取对应挂起位。没有挂起就直接返回。

2

清除 PR

CLEAR_IT 向 PR 写 1，让这一请求结束。

3

调用回调

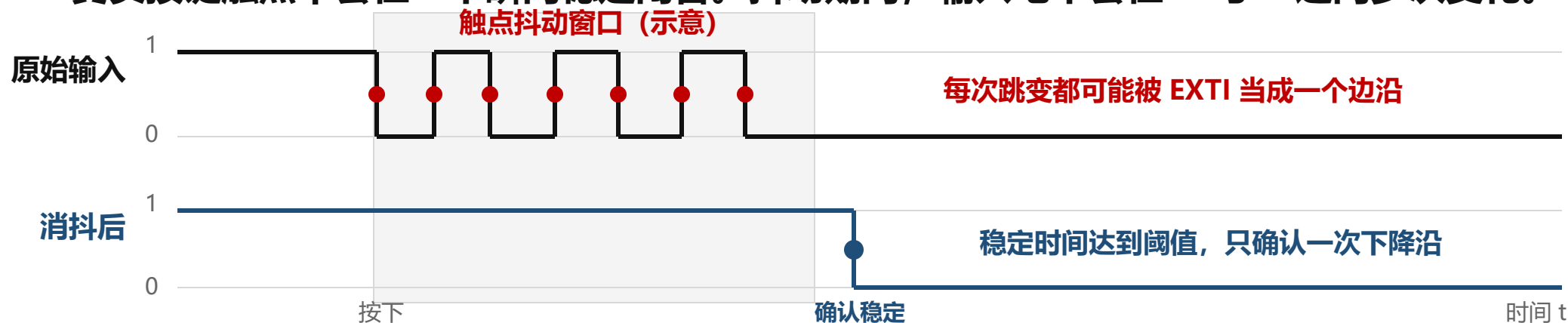
把 GPIO_Pin 交给用户代码判断和处理。

先清除再回调：如果回调执行期间又出现新边沿，PR 可以重新置 1，返回后还能再次进入中断。

HAL 的默认 Callback 是 `__weak`。用户实现同名函数后，链接器会采用用户版本。

按键抖动：一次机械动作，可能产生多个边沿

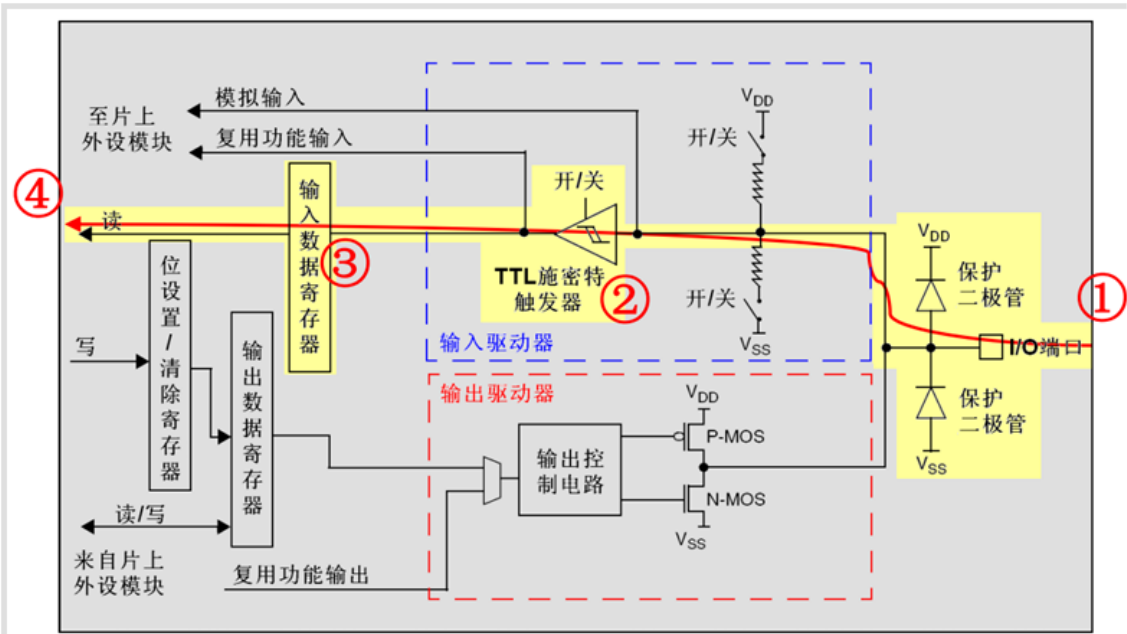
真实按键触点不会在一个瞬间稳定闭合。抖动期间，输入电平会在 0 与 1 之间多次变化。



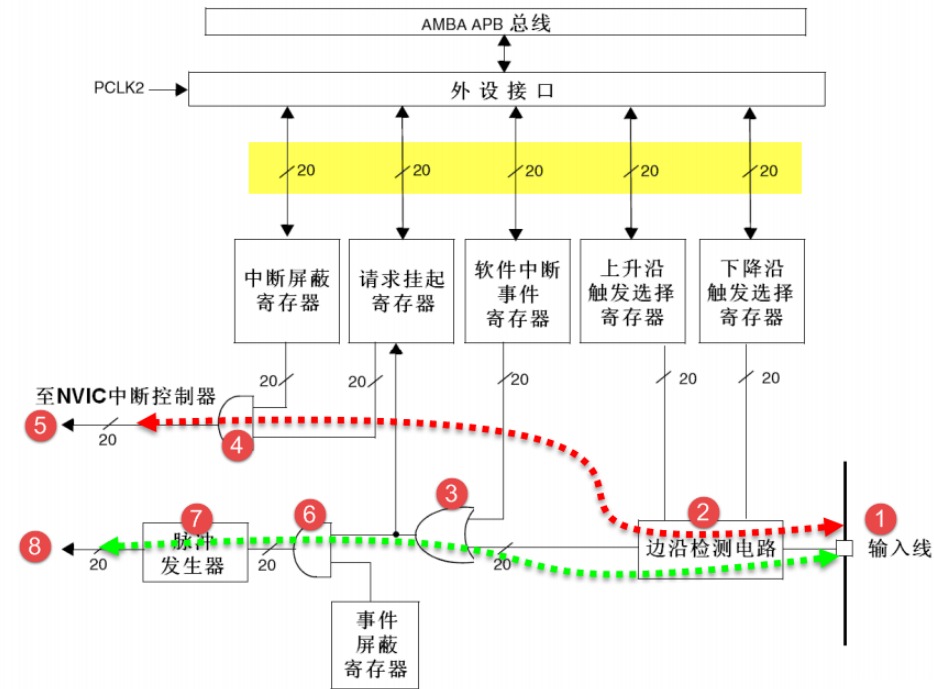
软件消抖思路：记录上次有效触发时间；在稳定窗口内到来的新边沿直接忽略。

- ISR 内只记录时间和状态，不做长延时
- 不要在高优先级 ISR 中调用 HAL_Delay()
- 对严格时序要求，可使用定时器或硬件 RC/施密特消抖

从引脚到回调：最后再看两张图



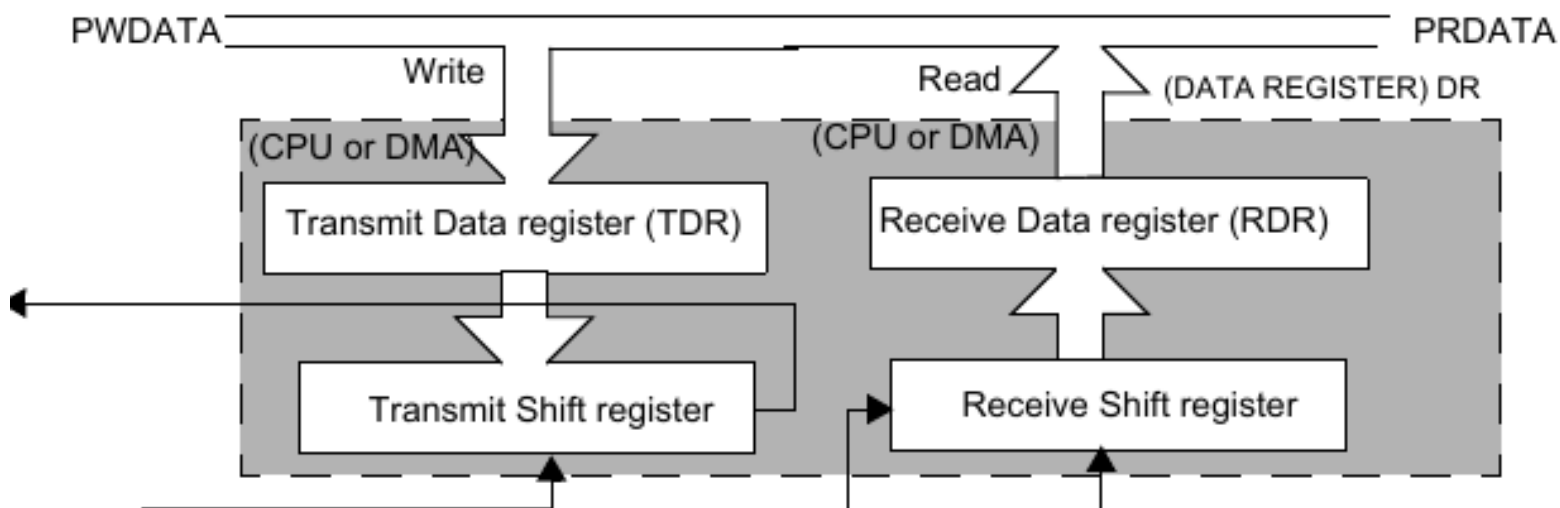
GPIO: 电压 → 施密特触发 → IDR / 片上外设



EXTI: 边沿 → PR → IMR → NVIC → ISR

USART: 通用同步/异步收发器

Figure 279. USART block diagram

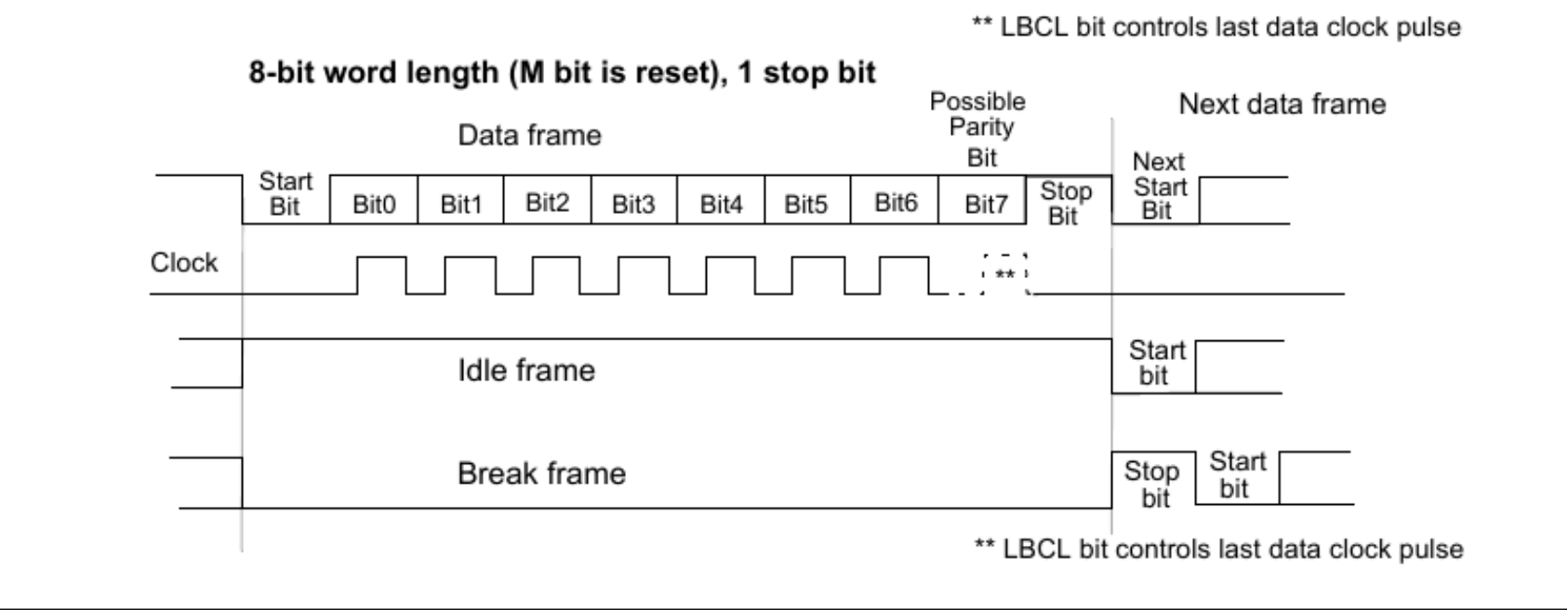


Universal Synchronous / Asynchronous Receiver / Transmitter

- 发送端把并行字节写入数据寄存器，再由移位寄存器逐位送到 TX
- 接收端从 RX 逐位采样，组成一个字节后交给 CPU 或 DMA
- TX 与 RX 可以同时工作，形成全双工通信
- 异步模式不输出 CK，同步模式额外输出时钟 CK

GPIO 提供电气入口和出口，USART 负责帧格式、移位和状态标志。

异步串口帧：双方必须使用同一套参数



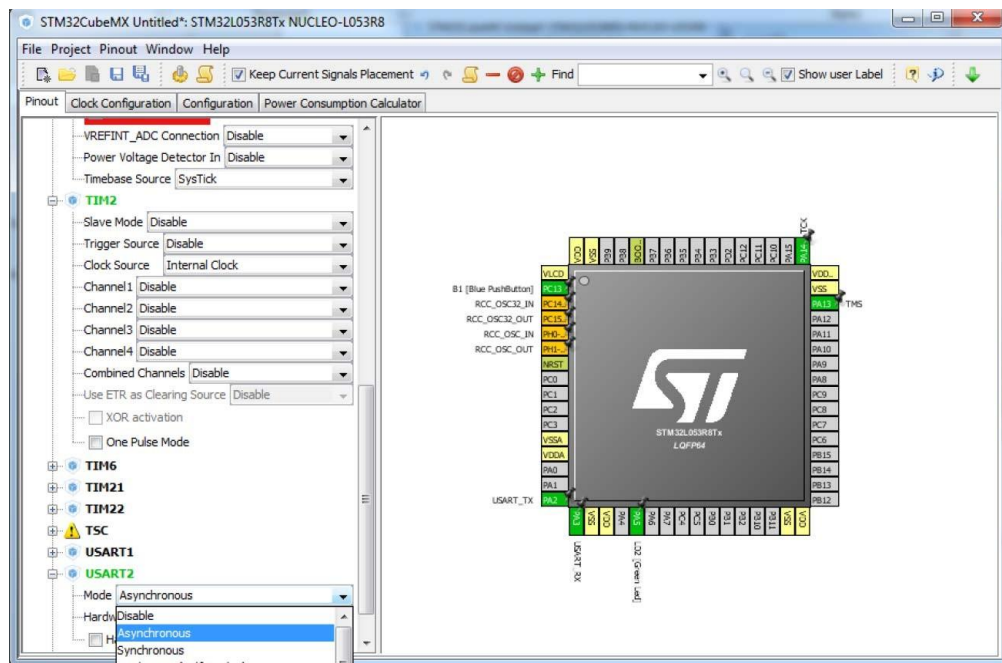
- 空闲状态：TX 保持高电平
- 起始位：一个低电平，提示新帧开始
- 数据位：8 位时通常先发送 Bit0
- N：不使用校验位
- 停止位：一个高电平，帧到此结束

异步模式不使用图中的 CK 线，只依据约定波特率采样。

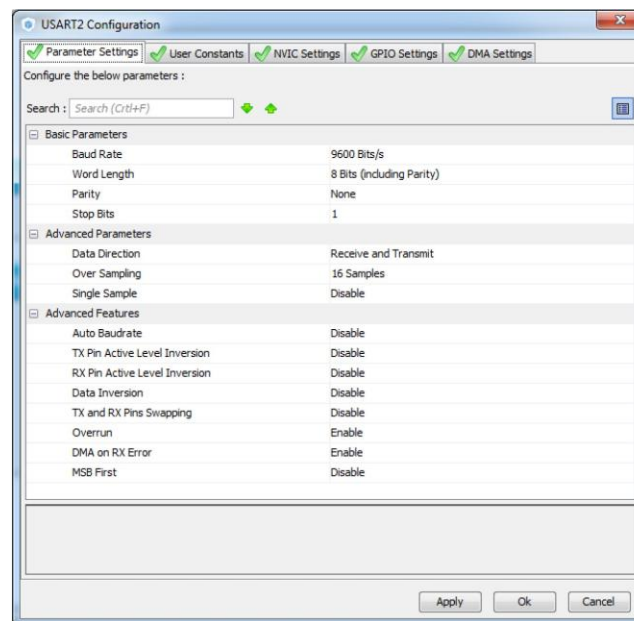
8N1 1 个起始位 + 8 个数据位 + 1 个停止位
 每发送 1 字节，线路实际传输 10 位

115200 bit/s 下，8N1 的单字节时间约 86.8 μs，理想有效速率约 11.52 kB/s。

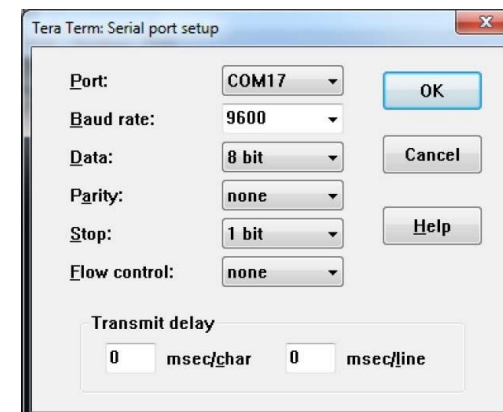
CubeMX: 先选择通信模式，再配置帧参数



① 选择 Asynchronous 或 Synchronous



② 设置波特率、字长、校验和停止位



③ 对端参数保持一致

异步串口通常使用 HAL_UART_Transmit_IT。同步模式使用 HAL_USART_Transmit_IT，并需要 CK 引脚。后续按指定的 HAL_USART_Transmit_IT 追踪中断发送过程。

阻塞发送与中断发送

阻塞发送

```
HAL_USART_Transmit(&husart1,  
                    msg, len, 1000);  
  
/* 函数内部持续等待 TXE / TC */  
next_task();
```

函数返回前已经完成全部发送

- 发送缓冲区必须保持有效，直到 HAL_USART_TxCpltCallback 执行
- 上一次传输尚未结束时，再次调用通常返回 HAL_BUSY
- USART 全局中断必须在 NVIC 中使能

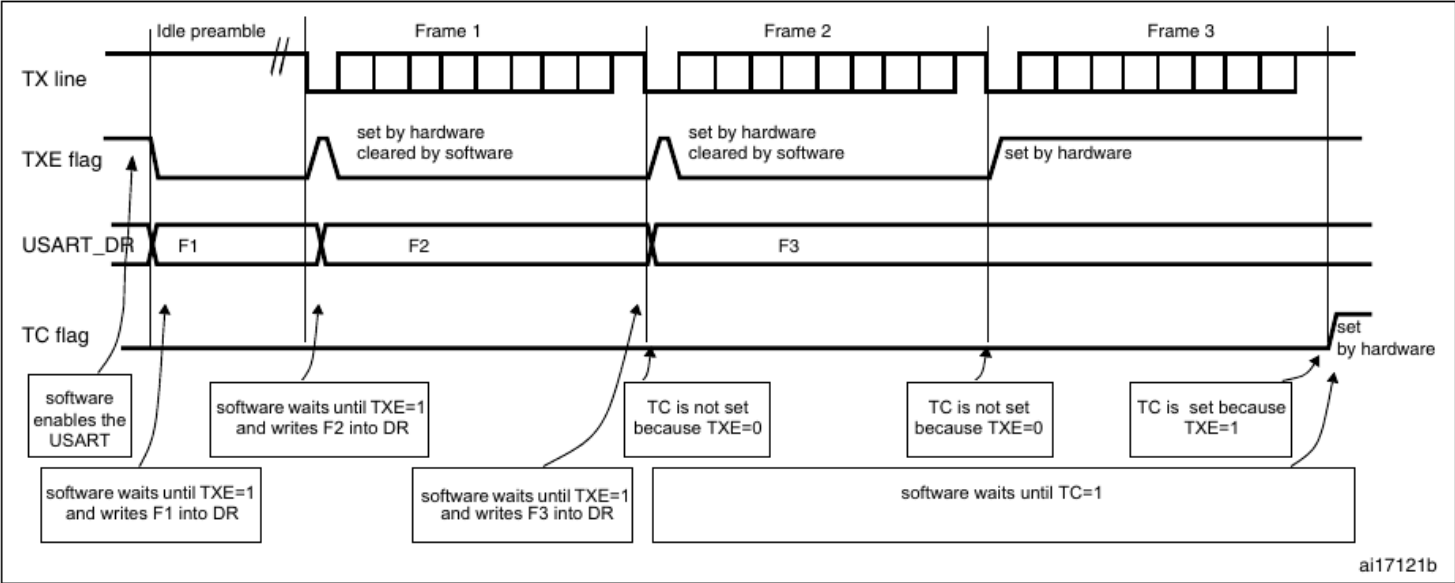
中断发送

```
HAL_StatusTypeDef st;  
st = HAL_USART_Transmit_IT(&husart1,  
                            msg, len);  
  
/* 启动成功后立即返回 */  
next_task();
```

函数只完成启动，后续字节由中断继续发送

USART 发送流水线：DR 与移位寄存器并行工作

Figure 282. TC/TXE behavior when transmitting



1 CPU 或 HAL 写 USART_DR

一个并行字节进入发送数据寄存器

2 硬件装入发送移位寄存器

DR 空出后 TXE 置 1，可以提前准备下一个字节

3 移位寄存器逐位驱动 TX

USART_BRR 决定位时间，帧格式由 CR1/CR2 配置

4 最后一个停止位结束

DR 与移位发送都完成后，TC 才置 1

USART_SR: TXE、TC 状态位 USART_CR1: TE、TXEIE、TCIE 使能位 USART_DR: 待发送数据

两级缓冲让上一字节仍在 TX 上移位时，CPU、DMA 或中断已经能把下一字节写入 DR，连续帧之间不会被软件等待拉出空隙。

HAL_USART_Transmit_IT: 先保存任务，再打开 TXE 中断

```
if (husart->State != HAL_USART_STATE_READY)
    return HAL_BUSY;

husart->pTxBuffPtr = pTxData;
husart->TxXferSize = Size;
husart->TxXferCount = Size;
husart->State = HAL_USART_STATE_BUSY_TX;

SET_BIT(husart->Instance->CR1, USART_CR1_TXEIE);
return HAL_OK;
```

1

检查 State

READY 才接受新任务，
BUSY 时返回 HAL_BUSY

2

保存指针与长度

HAL 只保存地址，不会复制
整段数据

3

记录剩余数量

TxxferCount 每发送一个数
据元素减 1

4

设置 BUSY_TX

阻止同一句柄同时启动第二
次发送

5

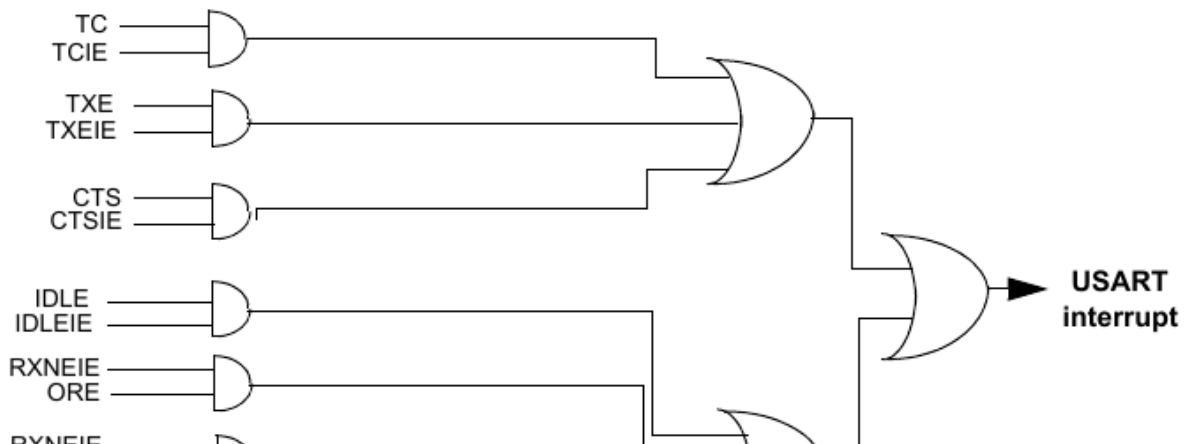
打开 TXEIE

数据寄存器为空时触发第一
次 USART 中断

HAL_OK 表示发送任务已启动。物理线路完成发送时，HAL 才调用 TxCpltCallback。

TC 中断：最后一个停止位结束后进入完成回调

Figure 302. USART interrupt mapping diagram



```
if ((isrflags & USART_SR_TXE) &&
    (cr1its & USART_CR1_TXEIE))
    USART_Transmit_IT(husart);

if ((isrflags & USART_SR_TC) &&
    (cr1its & USART_CR1_TCIE))
    USART_EndTransmit_IT(husart);
```

```
USART_EndTransmit_IT():
    关闭 TCIE
    State = READY
    HAL_USART_TxCpltCallback(husart)
```

- USART1_IRQHandler 只有一个入口
- HAL_USART_IRQHandler 读取 SR 与 CR1 后判断请求来源
- 完成回调可设置 tx_done, 或启动队列中的下一包

同一个 USART IRQ 可以承载 TXE、TC、RXNE 和错误等多种请求。

按键 EXTI 触发一次 USART 中断发送

```
static uint8_t msg[] = "KEY\r\n";
volatile uint8_t key_event;

void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
    if (pin == KEY_Pin) key_event = 1;
}

for (;;) {
    if (key_event &&
        HAL_USART_GetState(&husart1) ==
        HAL_USART_STATE_READY) {
        key_event = 0;
        HAL_USART_Transmit_IT(&husart1,
                               msg, sizeof(msg)-1);
    }
}

void HAL_USART_TxCpltCallback(USART_HandleTypeDef *h)
{
    tx_done = 1;
}
```

1

按键边沿

EXTI 记录挂起请求

2

EXTI IRQ

HAL_GPIO_EXTI_
IRQHandler()
清 PR 并分发

3

GPIO 回调

只设置 key_event, 尽快退出
中断

4

主循环

确认 USART READY 后启动
中断发送

5

USART IRQ

TXE 逐字节供数, TC 触发完
成回调

回调只记事件, 主循环再发送: 缩短中断占用。

连接 USB 串口终端并选择异步模式时, 将调用改为 HAL_UART_Transmit_IT(&huart1, ...).

USART 中断与 EXTI 中断的共同结构

比较项	USART 中断发送	GPIO 外部中断
请求来源	TXE、TC 等状态位置位	引脚边沿使 EXTI_PR 置位
外设使能	CR1.TXEIE、CR1.TCIE	EXTI_IMR 对应位
NVIC 通道	USART1_IRQn	EXTI0_IRQn 或共享通道
IRQHandler	USART1_IRQHandler 调用 HAL_USART_IRQHandler	EXTI0_IRQHandler 调用 HAL_GPIO_EXTI_IRQHandler
用户回调	HAL_USART_TxCpltCallback	HAL_GPIO_EXTI_Callback

两条链都经过请求来源、外设使能、NVIC、入口函数和 HAL 回调。下一节只需替换请求来源。